

SALMAN FARSI UNIVERSITY OF KAZERUN

Faculty of Engineering
Department of Computer Engineering

Bachelor of Science Thesis in Computer Engineering

A Unified Approach to Network Design, Purpose-Driven Automation of Operational Scenarios with Ansible, and Monitoring Implementation with Zabbix in a Small Organization

*A thesis submitted in partial fulfillment of the requirements
for the degree of Bachelor of Science in Computer Engineering*

Author

Mohammad Hosein Sheikhi

Supervisor

Dr. Parham Arjmand

June 2026 (Academic Year 2025–2026)

Declaration of Authorship

I, Mohammad Hosein Sheikhi, declare that this thesis titled “A Unified Approach to Network Design, Purpose-Driven Automation of Operational Scenarios with Ansible, and Monitoring Implementation with Zabbix in a Small Organization” and the work presented in it are my own.

I confirm that:

- This work was carried out independently as part of my Bachelor of Science studies at Salman Farsi University of Kazerun.
- Where I have consulted the published work of others, this is always clearly attributed and cited.
- Where I have quoted from the work of others, the source is always given; with the exception of such quotations, this thesis is entirely my own work.
- All principal sources of help have been acknowledged.

Mohammad Hosein Sheikhi

June 2026

Dedication

To my parents, whose unwavering support made this work possible, and to my grandparents,
whose wisdom and love have always been my guiding light.

Acknowledgements

I would first like to express my sincere gratitude to my supervisor, Dr. Parham Arjmand, for his valuable guidance and support throughout this thesis.

I am deeply thankful to my family for their constant support, patience, and encouragement during my studies — none of this would have been possible without them — and to my friends, especially Mr. Amin Salehi Farsani, for standing by me throughout this work.

Finally, my appreciation goes to the open-source communities behind Zabbix, Docker, and Ansible. The tools, documentation, and collective knowledge they share so freely formed the very foundation of this project and continue to make modern, accessible engineering possible.

Abstract

As computer networks in enterprise environments continue to grow in size and complexity, the traditional approach to managing them—relying largely on manual configuration through the command-line interface (CLI)—can no longer satisfy the requirements that matter most: scalability, agility, and reliability. In addition to being slow, manual management is a leading cause of human error, which in turn leads to unexpected outages across the infrastructure [1]. This thesis presents a comprehensive, unified, and modern approach to infrastructure management through the design, deployment, automation, and monitoring of a computer network built around enterprise-grade standards. The project is organized into three principal phases. In the first phase, the network was designed in a structured manner using hierarchical models, and core services such as DHCP and DNS were deployed on Windows Server within the GNS3 emulator. In the second phase, using the Infrastructure as Code (IaC) paradigm and the open-source Ansible platform, device configuration was fully automated without requiring any software agent (agentless). Finally, in the third phase, the Zabbix monitoring system was deployed for real-time monitoring of resources, traffic, and events, ensuring that the network remained stable and observable. The results demonstrate that combining sound design with automation and centralized monitoring yields a significant reduction in service-deployment time, eliminates manual configuration errors, and improves operational efficiency throughout the network life cycle.

Keywords: Network Automation; Infrastructure as Code (IaC); NetDevOps; Ansible; Zabbix; Network Monitoring; Observability; SNMP; Docker; GNS3; Configuration Management.

Table of Contents

Chapter One: Introduction and Project Overview	1
1.1. Introduction	1
1.2. Problem Statement.....	1
1.3. Project Objectives.....	2
1.4. Thesis Structure	2
Chapter Two: Literature Review and Foundational Concepts	4
2.1. Chapter Introduction.....	4
2.2. Computer Network Architecture and Design	4
2.2.1. The Hierarchical Network Model.....	4
2.2.2. Virtual LANs (VLANs) and Related Protocols.....	5
2.3. GNS3 Network Simulation and Emulation Environments.....	5
2.3.1. The Difference Between Simulation and Emulation.....	5
2.3.2. The Role of GNS3 in This Research	6
2.4. The Evolution of Network Management: Moving to Infrastructure as Code (IaC)	6
2.4.1. The Concept of Infrastructure as Code.....	6
2.4.2. The Idempotency Principle.....	7
2.5. The Ansible Automation Platform and Agentless Architecture.....	7
2.5.1. The Communication Mechanism and the Push Model.....	7
2.5.2. The Logical Structure: Playbooks and Modules	7
2.6. Comprehensive Monitoring Systems and an Introduction to Zabbix.....	8
2.6.1. The Zabbix System Architecture	8
2.6.2. Data Collection Mechanisms.....	8
2.7. Centralized Infrastructure Services (Windows Server: DHCP & DNS)	9
2.7.1. Dynamic Host Configuration Protocol (DHCP).....	9

2.7.2. Domain Name System (DNS)	9
Chapter Three: Phase One – Network Infrastructure Architecture, Design, and Deployment .	10
3.1. Chapter Introduction.....	10
3.2. Network Topology Analysis and IP Addressing Scheme	11
3.3. Data Link Layer Strategy: Managing VLANs and Layer 2 Security	12
3.3.1. Separating Broadcast Domains.....	13
3.3.2. Securing the Native VLAN	13
3.3.3. Implementing the Switch Virtual Interface (SVI)	13
3.4. Physical and Data Link Layer Challenges: Troubleshooting a Duplex Mismatch.....	13
3.5. Layer 3 Architecture and Centralizing the Exit Gateways	14
3.5.1. Implementing Switch Virtual Interfaces (SVIs) on swDistribution	14
3.5.2. Dynamic Routing with OSPF	14
3.5.3. Securing the OSPF Control Plane with Passive Interfaces	15
3.6. Connecting to the Global Network and Managing the Core/Edge Router	16
3.6.1. Configuring Network Address Translation (NAT/PAT) on the Edge Router.....	16
3.6.2. Centralized Default-Route Origination on the Edge Router	17
3.7. Deploying Processing Nodes and Infrastructure Servers in the Management VLAN ...	17
3.7.1. Ansible Controller Server (Ubuntu Server).....	17
3.7.2. Microsoft Infrastructure Server (Windows Server 2022).....	18
3.8. Stability and Integrity Analysis (Post-Implementation Analysis).....	18
Chapter Four: Phase Two – Network Automation Architecture and Code-Based Configuration Deployment	19
4.1. Chapter Introduction.....	19
4.2. Moving from Interactive Management to a Declarative Approach.....	19
4.3. Preparing and Architecting the Control Node	19

4.3.1. The Critical Role of the Python Interpreter	20
4.3.2. The Secure Transport Mechanism	20
4.4. The Connection Layer Architecture for Network Devices (Network Connection Plugins)	20
4.4.1. The Network_CLI Plugin	20
4.5. Designing the Data Model: The Inventory System	21
4.5.1. Designing the Inventory Data Model and Explaining the Interactive Behavior Variables	21
4.5.2. Breaking Down the Behavioral Parameters and Secure Sessions in the Automation Layer	22
4.5.3. Logical Grouping and Variable Hierarchy	23
4.6. Breaking Down the Code Architecture and Implementing the Operational Scenarios ..	23
4.7. Scenario One: A Dynamic, Automated Backup System (Dynamic Backup Automation)	24
4.7.1. Workflow Analysis	24
4.8. Scenario Two: Implementing a Baseline and Standardization (Baseline Standardization)	25
4.8.1. Declarative State Management	25
4.8.2. Idempotency in Action	26
4.9. Workflow Integration and the Execution Logic of the Scenario Hierarchy	27
4.10. Scenario Three: Provisioning Logical Infrastructure and Data-Driven Management of Virtual Networks	27
4.10.1. Removing VTP and Moving the Source of Truth	27
4.10.2. Loop Architecture and Separating Data from Logic	28
4.11. Scenario Four: Security Hardening and Reducing the Attack Surface	28
4.11.1. Stopping Vulnerable Services and Controlling Discovery Protocols	29

4.11.2. Securing Remote Access Lines (VTY & Console Lines)	29
4.12. Scenario Five: Instrumentation and Preparing the Monitoring Foundation	30
4.12.1. Unified Configuration of SNMP	31
4.13. Analyzing the Execution Output and Validating the Desired State	32
4.13.1. Play Recap Analysis	32
4.13.2. Transport-Layer Error Handling.....	32
Chapter Five: Phase Three – Centralized Monitoring Architecture, Performance Management, and Building Observability.....	33
5.1. Chapter Introduction.....	33
5.2. Comparative Evaluation of Infrastructure Monitoring Platforms	33
5.3. The Containerization Paradigm: Rationale for Migrating to Docker	34
5.3.1. What Is Docker, and How Does It Differ from Traditional Virtualization?	34
5.3.2. The Benefits of Deploying Zabbix on Docker in This Project.....	34
5.4. Local Deployment Challenges: Circumventing International Sanctions and Migrating to Domestic Mirrors.....	35
5.4.1. Technical Analysis of the Docker Hub Blockage	35
5.4.2. The Engineering Solution: Configuring Docker Registry Mirrors.....	35
5.5. Data Infrastructure Architecture and Ensuring Data Persistence	36
5.5.1. Choosing and Examining the Database in the Container Environment	36
5.5.2. Implementing Docker Volumes for Data Persistence	36
5.6. Analyzing the Structural Link Between the Automation and Monitoring Phases	37
5.6.1. The Role of Ansible's Fifth Scenario in Enabling Monitoring.....	37
5.6.2. The Engineering Benefit of Eliminating Configuration Drift	37
5.7. Engineering the Centralized Dashboard and Analyzing Live Metrics	37
5.8. Examining the Telemetry Transport Layer: Why SNMP?	38
5.8.1. SNMP's Information Architecture: The MIB and OID Concepts	38

5.9. The Operational Monitoring Subsystem and Node Availability Analysis	39
5.9.1. Analyzing the Availability Indicator Mechanism	39
5.10. Dynamic Monitoring with Templates and the Magic of LLD.....	40
5.10.1. The Concept of Inheritance in Zabbix Templates	40
5.10.2. Examining the Low-Level Discovery (LLD) Mechanism	41
5.11. Examining the Core/Edge Router's Operational Parameters (Router 1 Live Parameters Analysis).....	41
5.11.1. Breaking Down R1's Monitored Metrics.....	41
5.12. The Intelligent Event Analysis Layer: Trigger Architecture and Alert Thresholds	42
5.12.1. Explaining the Logic of the Conditional Expressions and Math Functions in Triggers.....	42
5.12.2. The Alert Flapping Problem and Applying Hysteresis	43
5.13. Classifying Event Severity Levels (Severity Levels Mapping)	43
5.14. Alerting Mechanisms and the Escalation Process	44
5.14.1. Escalation Path Architecture and Alerting Design	44
5.14.2. The Interactive Notification Platform in Domestic Infrastructure	44
5.15. Examining the Container Orchestration: Deploying Zabbix on Docker	44
5.15.1. Analyzing the Docker-Compose File Structure and Defining the Services	45
5.15.2. Technical Analysis of the Deployment Parameters.....	46
5.15.3. The Port Forwarding Architecture and Accessing the Zabbix UI.....	46
5.16. Visual Monitoring and Designing Interactive Network Maps (Zabbix Network Maps)	47
5.16.1. Drawing the Topology and Binding It to Data Elements	47
5.16.2. Dynamic Link Indicators	47
5.17. Big-Data Management Strategy and the Storage Life Cycle	48
5.17.1. Separating the History and Trends Architecture	48

5.18. A Scalable Expansion Strategy and Distributed Monitoring Architecture.....	49
5.18.1. The Challenges of Centralized Monitoring Architecture	49
5.18.2. The Conceptual Implementation of the Zabbix Proxy as a Middle-Layer Solution	49
5.19. Critical Analysis and Summary of the Monitoring Phase's Achievements	50
Chapter Six: Conclusion, Technical Evaluation, and Future Development Horizons	51
6.1. Chapter Introduction.....	51
6.2. A Matrix-Based, Step-by-Step Evaluation of the Three Project Phases	51
6.2.1. Analyzing Phase One's Performance: Reinforcing the Network Backbone.....	52
6.2.2. Analyzing Phase Two's Performance: Leaping to the Infrastructure as Code (IaC) Paradigm.....	53
6.2.3. Analyzing Phase Three's Performance: Completing the Network Life Cycle with Observability	53
6.3. Analyzing the Engineering Achievements and Innovations.....	53
6.4. Analyzing the Geopolitical Challenges, Sanctions, and Barriers to Deploying Open- Source Infrastructure	54
6.4.1. Re-Engineering the Container Registry in an Isolated Environment	54
6.5. How the Emulator Architecture (GNS3) Converges with Physical Production Environments.....	55
6.5.1. Examining the ASIC Hardware Layer versus CPU-Bound Processing	55
6.5.2. Technical Root-Cause Analysis of the Redundancy Protocol Bug (HSRP/VRRP Flapping).....	55
6.6. Risk Assessment, Threat Analysis, and Infrastructure Security Hardening Strategies ..	56
6.6.1. Automating Device Access-Layer Security with Ansible.....	57
6.6.2. Vulnerability Analysis of the Monitoring Layer (SNMPv2c Mitigation).....	57
6.7. Quantitative and Qualitative Analysis of the Monitoring Metrics and Operational Efficiency.....	58

6.7.1. Formulating and Analyzing the MTTR and MTBF Metrics	58
6.8. Future Development Horizons, Part One: Implementing GitOps in Network Infrastructure Management.....	59
6.8.1. Explaining the Single Source of Truth Concept.....	60
6.8.2. Orchestrating an Automated Validation Pipeline (CI/CD Pipeline)	60
6.8.3. Technical Analysis of the GitOps Pipeline Steps.....	61
6.9. Future Development Horizons, Part Two: AI in Network Monitoring and Event Management (AIOps).....	61
6.9.1. Dynamic Behavioral Monitoring Based on Machine Learning Algorithms (Baseline Detection)	62
6.10. The Knowledge Management Approach, Code Structuring, and Engineering Documentation	62
6.10.1. Self-Documenting Infrastructure with Declarative Code.....	62
6.11. Technical-Economic Analysis: Total Cost of Ownership (TCO) and Return on Investment (ROI).....	63
6.11.1. Total Cost of Ownership (TCO) Analysis.....	63
6.11.2. Return on Investment (ROI) Calculations	63
6.12. Final Summary, High-Level Evaluation of the Achievements, and Closing Words....	64
References	66

List of Figures

Figure 3.1. Logical and physical network topology in the GNS3 environment.....	11
Figure 5.1. The centralized Zabbix monitoring dashboard and the stability status of the network nodes	38
Figure 5.2. High-level communication architecture of SNMP and the data flow between the manager and the agent	39
Figure 5.3. Configuration of the network devices' inventory database in Zabbix	40
Figure 5.4. Live charts of interface traffic and hardware-resource usage on router R1	42
Figure 5.5. The dynamic, interactive Zabbix network map with live key performance indicators for continuous monitoring of the organization's network nodes	48
Figure 6.1. The layered security-hardening and defense-in-depth model implemented in the project	56
Figure 6.2. The GitOps pipeline workflow for automatically orchestrating network-device configuration.....	60

List of Tables

Table 3.1. IP address allocation matrix and the organization's logical VLAN structure..	12
Table 5.1. Event severity levels, standard color codes, and their operational meaning in the Zabbix monitoring system	43
Table 6.1. Strategic objectives matched against the operational achievements of the three project phases	52
Table 6.2. Infrastructure-management metrics: the traditional model versus the modern model	59
Table 6.3. Financial comparison of the NetDevOps platform versus proprietary models	63

Chapter One: Introduction and Project Overview

1.1. Introduction

Over recent decades, computer networks—as the foundational medium for organizational data exchange and service delivery—have expanded at an unprecedented rate, both in physical scale and in the diversity of logical services they carry. The emergence of paradigms such as cloud computing, the Internet of Things (IoT), and big data has placed additional pressure on communication infrastructure. In such a context, network administrators and infrastructure engineers require approaches capable of delivering highly available networks with the lowest possible latency and disruption.

Although network hardware such as routers and switches has advanced considerably, the methods used to manage and configure these devices have remained largely unchanged for years. The traditional approach to network management relies on the administrator connecting directly to each individual device through protocols such as SSH or Telnet and entering commands manually. With the advent of modern architectures and the need for continuous, rapid change—particularly in DevOps-oriented environments—this approach has become a significant bottleneck [2]. Consequently, the transition toward software-defined networking (SDN) and network automation has become essential in computer engineering and information technology. Recognizing this, the present project models the complete life cycle of building a network—from inception (design) through rollout (automation) to maintenance (monitoring)—in a practical manner aligned with current international standards.

1.2. Problem Statement

One of the foremost challenges facing enterprise networks today is change management and the preservation of configuration consistency. Studies of service failures consistently identify human error as one of the primary causes of unplanned network outages in enterprise environments [3]. Moreover, traditional networks provide no centralized view of network health, meaning that administrators typically become aware of a problem only after a failure has occurred and service quality has degraded—a reactive approach.

Accordingly, the central question this research seeks to answer is: “How can open-source automation and monitoring tools be integrated to optimize the deployment and maintenance of network infrastructure in a manner that reduces reliance on manual configuration while providing full visibility into the state of the network?” In the absence of an accurate monitoring system and of automation, skilled personnel expend time on repetitive tasks, and the organization’s operating expenditure (OpEx) rises sharply.

1.3. Project Objectives

The principal aim of this project is to design and implement a modern architecture for managing network infrastructure, founded on three pillars: standardized design, configuration automation, and continuous monitoring. The specific objectives of this thesis are as follows:

- Designing scalable infrastructure: implementing a layered network design model (Access, Distribution, Core) and deploying basic network services such as DHCP and DNS on Windows Server.
- Implementing code-based automation (IaC): employing the Ansible framework to develop playbooks that configure network devices (switches and routers) automatically, repeatably, and without error.
- Eliminating the human-error bottleneck: standardizing the configuration process through the centralized management of YAML files.
- Improving network observability: deploying the Zabbix monitoring system to collect data in real time, monitor bandwidth, port status, and server resources, and establish proactive alerting.
- Simulating an enterprise environment: constructing an isolated testbed in GNS3 that closely emulates the behavior of real network equipment.

1.4. Thesis Structure

To address the objectives above comprehensively, this document is organized into six chapters. Following the present introductory chapter, Chapter Two reviews the literature and the foundational concepts, including network architecture, the principles of automation, an introduction to Ansible and Zabbix, and the concept of Infrastructure as Code. Chapter Three covers the first phase of the project—designing the network topology, configuring VLANs, and

deploying the Windows Server services. Chapter Four (Phase Two) addresses the deployment of Ansible, the development of playbooks, and the automation of the devices. Chapter Five (Phase Three) concerns the integration of the Zabbix monitoring platform and the manner in which the network elements are monitored. Finally, Chapter Six summarizes the results, discusses the technical challenges encountered, and offers recommendations for future development.

Chapter Two: Literature Review and Foundational Concepts

2.1. Chapter Introduction

Before proceeding to the practical phase and describing how the proposed architecture was constructed, it is worthwhile to establish a thorough understanding of the underlying concepts, protocols, and technologies employed in this work. This chapter examines the theoretical foundations of modern computer networks, the principles of logical and physical design, and the transition from traditional methods to newer paradigms such as Infrastructure as Code (IaC) and network automation. It also considers the principal tools and platforms used in the project—the GNS3 emulator, the Ansible automation framework, and the Zabbix monitoring platform—from a software-architecture perspective and in terms of their role in enterprise environments.

2.2. Computer Network Architecture and Design

Designing a network that is stable, scalable, and secure requires adherence to standard, proven engineering models. In large enterprise networks, the flat network design approach has become obsolete owing to problems such as excessive broadcast traffic, lack of flexibility, and the difficulty of troubleshooting [4]. Hierarchical models are used instead, partitioning the network into logical layers with clearly defined responsibilities.

2.2.1. The Hierarchical Network Model

Cisco, as one of the pioneers of the networking industry, introduced a three-layer design model that is now regarded as a de facto standard in infrastructure architecture. This model comprises three distinct layers:

- **Access Layer:** the point at which end users, systems, servers, and peripheral devices connect to the network. Here, access switches enforce basic security policies such as port security, assign VLANs, and manage device power through Power over Ethernet (PoE). The principal objective at this layer is to provide dedicated bandwidth and to isolate user traffic at Layer 2 (the data-link layer) of the OSI model.
- **Distribution Layer:** situated between the access and core layers, this layer is responsible for aggregating traffic from the access switches. The most important networking operations

occur here—inter-VLAN routing, traffic filtering based on access control lists (ACLs), QoS policies, and route summarization. In effect, the distribution layer constitutes the boundary that separates broadcast domains [5].

- Core Layer: the function of the core layer—the backbone of the network—is to forward packets as rapidly as possible with minimal delay. This layer avoids processing-intensive tasks such as ACLs and complex routing, so that the full processing capacity of the device can be devoted to high-speed switching. Reliability and redundancy are critical at this layer.

2.2.2. Virtual LANs (VLANs) and Related Protocols

The concept of the virtual local area network (VLAN) is one of the cornerstones of modern network design. VLANs make it possible to partition a single physical switch logically into several separate virtual switches. This logical separation confers a number of benefits: smaller broadcast domains, improved security through the isolation of traffic between departments, and easier resource management. To carry traffic from multiple VLANs between one another, network devices require trunk links and the IEEE 802.1Q encapsulation standard.

In small-scale deployments, however, it is common practice to consolidate the core function with another layer — most often the network edge — onto a single device, reducing cost and complexity while preserving the layer's logical role.

2.3. GNS3 Network Simulation and Emulation Environments

In infrastructure engineering projects, deploying a design directly onto physical equipment (a production environment) without prior testing is a high-risk undertaking. For this reason, the use of simulation environments to validate network architecture has become an industry standard.

2.3.1. The Difference Between Simulation and Emulation

It is important to distinguish between simulators (such as Cisco Packet Tracer) and emulators (such as GNS3). Simulators merely imitate the software behavior of devices within a limited environment and do not execute a real operating system. Emulators, by contrast, employ virtualization technologies such as Dynamips, QEMU, and KVM to run genuine network

operating systems (such as Cisco IOS, MikroTik RouterOS, or hardware firewalls) on the host computer's hardware [6].

2.3.2. The Role of GNS3 in This Research

GNS3 (Graphical Network Simulator-3) is a powerful platform for implementing complex networking concepts. In this project, GNS3 was not used merely to test simple connectivity; rather, it served as a complete testbed for integrating network devices (switches and routers) with x86-based virtual machines, such as the Windows Server and the Linux server (hosting Ansible and Zabbix). This integration causes the implemented scenario to behave in the same manner as a real-world scenario within an actual organization.

Although GNS3, as an advanced emulator, executes genuine network operating systems, the hardware limitations of the host CPU nevertheless affect device behavior. This tension between emulation and physical production environments, together with its impact on network protocols, is examined in detail in the technical evaluation presented in Section 6.5 of Chapter Six.

2.4. The Evolution of Network Management: Moving to Infrastructure as Code (IaC)

The traditional approach to network management—manual configuration through the CLI—does not scale as networks grow. Under this approach, engineers must log in to each device individually to effect even a simple change, which is not only slow but also greatly increases the likelihood of human error arising from mistyped commands or omitted settings.

2.4.1. The Concept of Infrastructure as Code

The information technology industry's response to this challenge was Infrastructure as Code (IaC). IaC is a process in which physical and virtual infrastructure is provisioned and managed through machine-readable code rather than through manual, interactive steps. Within this paradigm, network configuration is treated as software code—meaning that it can be stored in version-control systems (such as Git), its change history can be tracked, and CI/CD processes can be applied to it [7].

2.4.2. The Idempotency Principle

One of the most important characteristics of IaC systems is idempotency. It guarantees that, regardless of how many times a configuration is applied to a device, the end result is always identical. If the current state of the device already matches the desired state defined in the code, the system makes no changes whatsoever. This property is vital in network automation because it prevents the re-execution of configuration scripts on live equipment from causing disruption, and it preserves service stability. This approach forms the foundation for entering the domain of NetDevOps.

2.5. The Ansible Automation Platform and Agentless Architecture

Among configuration-management and automated-deployment tools, Ansible (developed by Red Hat) has earned a distinctive place in enterprise environments owing to its ease of learning and its distinctive architecture. Unlike tools such as Puppet or Chef, which require an agent to be installed on every target device, Ansible employs an agentless architecture [8].

2.5.1. The Communication Mechanism and the Push Model

Ansible's architecture is based on a push model: the central server (control node) pushes configurations to the network devices (managed nodes). On Linux machines and network devices, this connection is generally established over the secure SSH protocol, and on Windows machines over WinRM. Features such as SSH multiplexing make it possible to connect to hundreds of devices simultaneously with minimal processing overhead. In sensitive networks where the installation of third-party software on routers and switches is prohibited for security reasons, this constitutes a genuine competitive advantage.

2.5.2. The Logical Structure: Playbooks and Modules

Ansible's execution engine operates on a declarative approach. In files called playbooks (written in the human-readable YAML language), the user describes the device's desired end state, and it is Ansible's responsibility to compile and execute the commands required to reach that state, using purpose-built modules (such as `ios_config` for Cisco devices or `mikrotik_routeros` for MikroTik routers) [9]. This structure enables network engineers to express even the most

complex topologies as standardized code—without requiring deep programming knowledge—and to maintain them in version-control systems.

2.6. Comprehensive Monitoring Systems and an Introduction to Zabbix

In the network life cycle, once the design and deployment phases are complete, the monitoring and maintenance phase assumes critical importance. The purpose of monitoring is to move from a reactive approach to a proactive one, in which the system warns administrators of anomalies before a full outage occurs [21].

2.6.1. The Zabbix System Architecture

Zabbix is a unified, open-source, enterprise-class monitoring solution capable of tracking millions of metrics from networks, servers, virtual machines, and cloud services. The Zabbix architecture comprises several components:

- Zabbix Server: the central processing core that collects data (polling/trapping), evaluates critical conditions (triggers), and dispatches alerts.
- Zabbix Database: a relational database (such as MySQL or PostgreSQL) that holds all configuration settings, historical data, and events.
- Zabbix Frontend: the web-based interface used for data visualization, the rendering of graphs, and configuration management.
- Zabbix Proxy: in distributed networks, proxies collect data from local devices and return it to the central server in aggregate, thereby substantially reducing the load on the main server.

2.6.2. Data Collection Mechanisms

In this architecture, network devices are monitored primarily through SNMP (Simple Network Management Protocol). The use of the secure versions of this protocol (such as SNMPv3) permits the encryption and authentication of monitoring traffic. For Windows and Linux servers, the Zabbix Agent is employed, which collects operating-system-level data (such as processes, memory usage, and storage) with very high precision.

2.7. Centralized Infrastructure Services (Windows Server: DHCP & DNS)

No enterprise network can serve its clients effectively without basic addressing and name-resolution services. In this thesis, these critical services were deployed on Microsoft Windows Server in order to match production environments as closely as possible.

2.7.1. Dynamic Host Configuration Protocol (DHCP)

DHCP is responsible for the automatic assignment of IP addresses, gateway addresses, and network parameters to clients. The address-allocation process occurs through four basic steps, abbreviated as DORA (Discover, Offer, Request, Acknowledge). Deploying this service centrally on the server prevents IP conflicts and simplifies the management of the various subnets within the network's VLAN-based structure.

2.7.2. Domain Name System (DNS)

DNS functions as the directory of the network, translating human-readable domain names into the IP addresses the network uses. In environments where automation tools (such as Ansible) are to be deployed, a stable DNS server is critical, since automation scripts typically address devices by hostname or fully qualified domain name (FQDN) [8]. Nevertheless, a strategic engineering decision was made in this project's architecture: separating the infrastructure control plane from the name-resolution service.

To conserve processing resources, avoid circular dependencies, and ensure that the Ansible engine could continue to manage devices even in the event of a complete failure of the Windows (DNS) server, the infrastructure nodes in the inventory file are addressed directly by static IP addresses. The DNS service in this network is configured exclusively to handle communication for the departmental clients and access to web services, maintaining consistency at the user layer.

Chapter Three: Phase One – Network Infrastructure Architecture, Design, and Deployment

3.1. Chapter Introduction

Any enterprise-level automation or monitoring system depends fundamentally on a stable, structured, and scalable underlay network. This chapter describes the first phase of the project in detail—designing the topology, allocating logical resources, configuring the access and distribution layers, and deploying the basic services. In this implementation, the objective was to avoid unnecessary complexity at the physical layer and to concentrate on an optimal logical architecture, so that the network would not only isolate departments precisely but also be prepared to accept the Infrastructure as Code (IaC) scripts.

All implementation work described in this thesis was carried out in a controlled lab environment using the following toolchain: GNS3 2.2.54 as the network emulator, with a Cisco c7200 platform (IOS) used for the edge router and IOL-based switches (version 15.4.1) for the access and distribution layers; Ubuntu Server 22.04 LTS as the host operating system for the Ansible control node and the Docker host; Windows Server 2022 for the DHCP and DNS services; Ansible core 2.17.14 (Python 3.10.4, Jinja2 3.0.3) as the automation engine; and Docker 29.2.1 with Docker Compose v5.0.2 for the containerized Zabbix 6.4 monitoring stack. Specifying these versions allows the implementation to be reproduced precisely in future work.

3.2. Network Topology Analysis and IP Addressing Scheme

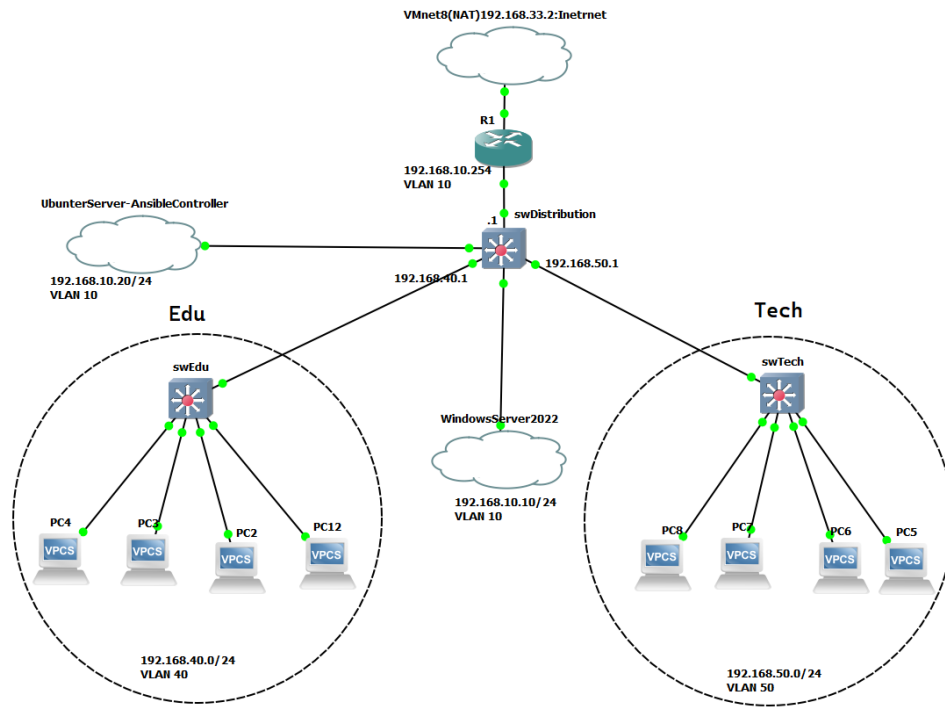


Figure 3.1. Logical and physical network topology in the GNS3 environment

As shown in the network schematic (the topology diagram), the architecture of this project is based on Cisco's three-layer hierarchical model, in which the access switches (swEdu, swTech) form the access layer, swDistribution serves as the distribution layer, and R1 consolidates the core layer's high-speed forwarding function with the network's internet-edge/gateway role onto a single device — a common cost-optimization in small enterprise deployments where a dedicated core switch is not economically justified.

In this scenario there are two principal departments—Education (Edu) and Technical (Tech)—each with its own traffic and security requirements. The IP plan was devised so that route summarization would be feasible subsequently and troubleshooting could be performed rapidly. The networks were allocated as follows:

- Management and Infrastructure – 192.168.10.0/24: hosts the critical servers, such as Windows Server 2022 at 192.168.10.10 and the Ubuntu-based Ansible controller at 192.168.10.20.
- Education Department – 192.168.40.0/24: connected to the swEdu access switch.

- Technical Department – 192.168.50.0/24: connected to the swTech access switch.
- Edge connection: using the VMnet8 network (192.168.33.0/24) in NAT mode for isolated communication with the global internet.

Table 3.1. IP address allocation matrix and the organization's logical VLAN structure

Device / Service	Interface / VLAN	IP Address	Subnet Mask	Gateway	Operational Role
Core/Edge Router (R1)	To VMnet8	192.168.33.x	Dynamic / NAT	192.168.33.2	Outbound to internet
Core/Edge Router (R1)	To swDist	192.168.10.254	255.255.255.0	—	Internal exit gateway
swDistribution	SVI - VLAN 10	192.168.10.1	255.255.255.0	192.168.10.254	Management gateway
swDistribution	SVI - VLAN 40	192.168.40.1	255.255.255.0	192.168.10.254	Education gateway
swDistribution	SVI - VLAN 50	192.168.50.1	255.255.255.0	192.168.10.254	Technical gateway
Ubuntu Server	VLAN 10	192.168.10.20	255.255.255.0	192.168.10.1	Ansible node / Docker host
Windows Server	VLAN 10	192.168.10.10	255.255.255.0	192.168.10.1	DNS and DHCP server
Education Clients	VLAN 40	192.168.40.0/24	255.255.255.0	192.168.40.1	PC2, PC3, PC4, PC12
Technical Clients	VLAN 50	192.168.50.0/24	255.255.255.0	192.168.50.1	PC5, PC6, PC7, PC8

3.3. Data Link Layer Strategy: Managing VLANs and Layer 2 Security

One of the most important architectural decisions in this project was restricting the access-layer switches (swEdu and swTech) to strict Layer 2 operation. Under this approach, the access switches do not engage in Layer 3 (IP) packet processing at all; they merely forward Ethernet frames [10]. This decision was made for several reasons:

- Reducing CPU and memory overhead on the user-edge devices.
- Concentrating security policies and ACLs on the distribution switch (swDistribution).
- Simplifying the network-automation process in subsequent phases.

3.3.1. Separating Broadcast Domains

To prevent traffic conflicts and improve security, the ports connected to workstations (PCs) were assigned manually (static access) to their respective VLANs. Clients PC2, PC3, PC4, and PC12 in the Education area were assigned to VLAN 40 (Edu), and clients PC5 through PC8 in the Technical department to VLAN 50 (Tech).

3.3.2. Securing the Native VLAN

In the 802.1Q protocol, frames belonging to the native VLAN traverse trunk links untagged. By default on Cisco devices, the native VLAN is set to VLAN 1, which poses a serious security risk in the context of VLAN-hopping attacks [11]. To eliminate this vulnerability, the native VLAN on all trunk links between the access and distribution switches was changed and reassigned to an unused blackhole VLAN, preventing any leakage of unmanaged data.

3.3.3. Implementing the Switch Virtual Interface (SVI)

Because the access-layer switches in this design lack routing capability, they required a management IP address for remote management over SSH (in preparation for deploying Ansible in the next phase). For this purpose, VLAN 10 was defined as the management network, and a switch virtual interface (SVI) was created on swEdu and swTech exclusively for VLAN 10, with management IP addresses assigned to them. In this way, user traffic (the data plane) was fully separated from management traffic (the control/management plane).

3.4. Physical and Data Link Layer Challenges: Troubleshooting a Duplex Mismatch

While the inter-switch links were being connected, one of the classic and significant problems in networking arose: a duplex mismatch. This occurs when auto-negotiation fails at both ends of a physical link—for example, when one end is set manually to full duplex while the other is in auto mode and, receiving no negotiation signals, reverts to the network default of half duplex [12].

Technical analysis and resolution. In this situation, the port configured for full duplex transmits frames without sensing the state of the line (carrier sense). At the same time, the half-duplex port—which follows CSMA/CD (Carrier-Sense Multiple Access with Collision Detection)—

interprets this simultaneous transmission as a collision or a late collision and discards the frames. This resulted in a sharp reduction in bandwidth, increased latency, and intermittent packet loss between the access and distribution switches. To address the root cause and maintain the stability of the trunk links carrying sensitive traffic, auto-negotiation was disabled on the uplink ports, and the speed and duplex were hard-coded to speed 1000 and duplex full at both ends. The result was fully stable Layer 2 connectivity.

3.5. Layer 3 Architecture and Centralizing the Exit Gateways

One of the fundamental challenges in designing multi-segment networks is determining where to place the default gateway for clients. In this project, in order to optimize routing and reduce the load on the access layers, a “centralized gateway on the distribution switch” architecture was adopted.

3.5.1. Implementing Switch Virtual Interfaces (SVIs) on swDistribution

Unlike traditional models that delegate routing to the Core/Edge Router, here the Layer 3 switch (swDistribution) serves as the network’s intelligence at Layer 3. For each VLAN defined in the previous phase (Edu, Tech, and Management), an SVI was created and assigned the IP address corresponding to its subnet [13]. This approach offered several technical benefits:

- Wire-speed switching: inter-VLAN routing is performed within the switch’s hardware (ASIC), keeping latency to a minimum.
- Reduced uplink traffic: traffic between users on a single distribution switch no longer needs to leave the switch and traverse a router (the router-on-a-stick method).
- Client stability: the access-layer clients (swEdu and swTech) regard the distribution switch as their gateway. Consequently, even if the link to the Core/Edge Router (R1) fails, internal communication (such as access to local servers) continues to function.

3.5.2. Dynamic Routing with OSPF

With inter-VLAN routing established on swDistribution, the internal subnets and the segment toward the Core/Edge Router still had to be made mutually reachable. Rather than maintaining static routes by hand—an approach that does not scale and grows error-prone as the network expands—Open Shortest Path First (OSPF) was adopted. OSPF was selected because it is an

open, vendor-neutral, industry-standard link-state protocol that aligns with the project's goal of avoiding vendor lock-in, and because its fast convergence and native support for classless addressing (VLSM) suit enterprise infrastructure [5].

Given the scale of the topology, a single-area design was chosen: all three internal subnets—Management (192.168.10.0/24), Education (192.168.40.0/24), and Technical (192.168.50.0/24)—were advertised into the OSPF backbone, area 0. A single-area configuration keeps the routing design simple and fully consistent for a network of this size, while leaving room to partition the domain into multiple areas should it grow in the future. With this configuration, swDistribution (OSPF router ID 1.1.1.1) and the Core/Edge Router R1 (OSPF router ID 2.2.2.2) establish an OSPF adjacency over the Management segment (VLAN 10) and exchange their routing information automatically, so that every Layer 3 device learns all internal subnets without a single manually written static route. This dynamic foundation is also what makes the centralized default-route origination described in Section 3.6.2 possible, since R1 injects its single default route into the same OSPF process for all peers to learn.

3.5.3. Securing the OSPF Control Plane with Passive Interfaces

Enabling a routing protocol on the VLAN interfaces introduces a subtle risk: by default, the Layer 3 switch transmits OSPF Hello packets out of every interface on which the protocol is active—including the SVIs that face the end-user departments (Education and Technical). Sending routing-protocol traffic toward access ports is both unnecessary and dangerous, since no legitimate router resides there. To address this, the passive-interface command was applied to the user-facing SVIs (passive-interface Vlan40 and passive-interface Vlan50). A passive interface still advertises its subnet into OSPF, but it no longer sends or processes Hello packets on that interface, so no neighbor adjacency can be formed through it.

This hardening measure delivers two concrete benefits. First, security: it prevents a malicious host on a user port from injecting forged OSPF packets to form a rogue adjacency and poison the routing table—an OSPF spoofing attack—thereby protecting the integrity of the control plane [17]. Second, efficiency: it eliminates the needless periodic multicast Hello traffic on the access segments, reducing control-plane overhead. The only interface left active for adjacency formation is the Management segment (VLAN 10), where the sole legitimate neighbor—router R1—resides.

The OSPF configuration was applied as a one-time, manual prerequisite during the initial device bring-up. Because routing-protocol parameters represent stable, infrastructure-level settings that are unlikely to change after initial deployment, they were intentionally kept outside the repeatable Ansible pipeline, which governs only the operational configuration scenarios described in Chapter Four.

3.6. Connecting to the Global Network and Managing the Core/Edge Router

In this topology, router R1 functions as the core layer of the network and simultaneously serves as the network edge (gateway of last resort) to the internet. Owing to the small scale of the organization, the core and the internet-edge roles were consolidated into a single device — a common and cost-effective design choice in small enterprise networks. It connects to the distribution switch through a Layer 3 link and, through another interface, to the VMnet8 network (the NAT environment within the hypervisor).

3.6.1. Configuring Network Address Translation (NAT/PAT) on the Edge Router

To complete the first phase and bring the network out of its isolated state, the deployment of address translation on the edge router (R1) was essential. Because this network's design uses private IP addresses, these cannot be routed directly over the internet.

To resolve this and enable the internal nodes (in particular the Ubuntu server, which must download software packages in later phases) to reach the global network, NAT overload (PAT – Port Address Translation) was configured on R1. In this configuration, the interface connected to the distribution switch was designated ip nat inside and the interface facing the internet (VMnet8) ip nat outside. Then, with a defined ACL, all outbound traffic from the internal subnets was translated to the IP address of the external interface. The CLI commands applied to R1 were as follows:

```
R1(config)# access-list 1 permit 192.168.0.0 0.0.255.255
R1(config)# interface Ethernet1/0
R1(config-if)# ip nat inside
R1(config)# interface FastEthernet0/0
R1(config-if)# ip nat outside
R1(config)# ip nat inside source list 1 interface FastEthernet0/0 overload
```

3.6.2. Centralized Default-Route Origination on the Edge Router

For every device in the network—the distribution switch and, through it, the end clients—to reach external destinations, the gateway of last resort had to be known across the entire routing domain rather than being hard-coded on each device one by one. To achieve this, R1 was made the single device that originates the default route. The only static route in the whole topology was the default route on R1 itself, pointing toward the VMnet8 NAT gateway (ip route 0.0.0.0 0.0.0.0 192.168.33.2). R1 was then instructed, by means of the default-information originate command within its OSPF process, to advertise this default route to all of its routing peers. As a result, swDistribution—and any further Layer 3 device added later—learned the path to the internet dynamically and automatically, without a single manually written static route on the internal switches. Because default-information originate is a standard feature of an open, vendor-neutral routing protocol, this approach propagates the gateway of last resort consistently across multi-vendor equipment and establishes a clean routing hierarchy in which traffic flows upward from the access layer to the distribution layer and finally out to the internet through the edge router. This design also centralizes control: should the internet uplink ever change, only R1's single static route needs updating, and the new path is then redistributed to the entire network instantly.

3.7. Deploying Processing Nodes and Infrastructure Servers in the Management VLAN

For centralized management and to prepare the automation and monitoring phases, two strategic servers were placed in the server farm (within VLAN 10). This VLAN was chosen in order to isolate management traffic from general user traffic.

3.7.1. Ansible Controller Server (Ubuntu Server)

A virtual machine running the Ubuntu GNU/Linux distribution was configured as the Ansible control node. Linux was selected for this role because automation tools are native to the Linux kernel and it handles SSH connections efficiently. This server was configured at 192.168.10.20, and the access required for it to reach all network devices (switches and routers) was ensured by the dynamic routing provided by OSPF across the Layer 3 domain.

3.7.2. Microsoft Infrastructure Server (Windows Server 2022)

This server was deployed to provide directory, DNS, and DHCP services on the network.

- DHCP service and the relay-agent mechanism: because the server resides in VLAN 10 yet must assign addresses to clients in VLAN 40 and VLAN 50, the IP helper-address feature was configured on the distribution switch's SVIs. This command converts the clients' broadcast requests into unicast and forwards them to the Windows Server [14].

3.8. Stability and Integrity Analysis (Post-Implementation Analysis)

After the configurations described above were complete, the network entered stress and stability testing. Using traffic-analysis tools (such as Wireshark within GNS3), the correct flow of packets between the various VLANs and through the NAT layer was verified. The results showed that placing the gateways at the distribution layer qualitatively improved round-trip time and left the network fully prepared to accept the complex NetDevOps scenarios of the subsequent phases.

Chapter Four: Phase Two – Network Automation Architecture and Code-Based Configuration Deployment

4.1. Chapter Introduction

As IT infrastructure continues to expand and organizations face an urgent need for agility in service delivery, the traditional CLI-based approach to network management has become an operational bottleneck. In the previous chapter, the network’s communication foundation was established in accordance with hierarchical design principles. This chapter represents the turning point of the research—the juncture at which the software-defined networking (SDN) paradigm and DevOps engineering practices converge to give rise to the concept of NetDevOps. In this phase, the Ansible automation platform is established as the network’s control plane, and configuration management of the access- and distribution-layer devices is transferred from static, manual work to a fully automated, repeatable, text-file-based structure (Infrastructure as Code).

4.2. Moving from Interactive Management to a Declarative Approach

In traditional network management, the engineer adopts an imperative approach, entering commands step by step to achieve a goal. Ansible, by contrast, is based on a declarative approach. Here, the user describes only the device’s “desired end state” (for example, that a specific VLAN with a specific name should exist), and Ansible’s engine determines the difference between the device’s current state and the desired state, issuing only the commands required to close that gap [15]. This property—termed idempotency—prevents destructive or redundant changes from occurring when scripts are re-executed, and maintains the stability of the network.

4.3. Preparing and Architecting the Control Node

In Ansible’s client–server architecture, the system from which the scripts are executed is termed the control node. In this project, a virtual machine running Ubuntu Server (within the management network at 192.168.10.20) was assigned this role.

4.3.1. The Critical Role of the Python Interpreter

Unlike Windows-based operating systems, Linux distributions support automation natively. At its core, Ansible is built with Python. The execution of the network modules and the processing of YAML files depend entirely on the Python runtime environment on the control node. In this implementation, before the Ansible core was installed, the base Python libraries and the Python package manager (pip) were updated in order to ensure full compatibility with the Cisco network modules [16].

4.3.2. The Secure Transport Mechanism

Because Ansible employs an agentless architecture, there is no need to install any intermediary software on the Cisco routers and switches (the managed nodes). Communication between the Ubuntu server and the network devices is established exclusively over the secure application-layer protocol SSH. At the code level, Ansible uses low-level Python libraries such as Paramiko to manage SSH connections with minimal processing overhead [8].

4.4. The Connection Layer Architecture for Network Devices (Network Connection Plugins)

One of the fundamental differences between automating network devices and automating Linux servers lies in the nature of network operating systems such as Cisco IOS. On Linux servers, Ansible transmits Python code directly to the target server and executes it there (the ssh plugin). Traditional network devices, however, lack a Python interpreter.

4.4.1. The Network_CLI Plugin

To overcome this limitation on network devices, Ansible employs a specialized connection plugin called `network_cli`. In this research, the `ansible_connection: network_cli` parameter was set in the configuration. Under this approach, rather than transmitting Python code to the switches, the control node opens an SSH connection at the terminal (CLI) level, parses the device's textual output (the prompt), and issues the corresponding Cisco commands natively. The `ansible_network_os: ios` parameter explicitly instructs the engine to interact with Cisco's syntax and behavior, thereby preventing command-syntax errors.

4.5. Designing the Data Model: The Inventory System

The first operational step in executing playbooks is to define the devices that are to be managed. This information is stored in a text-based database called the inventory.

4.5.1. Designing the Inventory Data Model and Explaining the Interactive Behavior Variables

To present the network devices to the Ansible control node, a centralized inventory file in the standard INI format, named `inventory.ini`, was developed. In this data structure, the devices are partitioned into separate groups according to their operational role and their position in the topology: `[routers]`, `[core_switches]`, and `[access_switches]`. It should be noted that the label `[core_switches]` denotes the operational role of `swDistribution` as the Layer 3 core of the switched fabric — the central switching node through which all inter-VLAN traffic passes — whereas the R1 router, grouped under `[routers]`, fulfils the routing-core and internet-edge role described in Chapter Three. These group labels therefore reflect each device's function within the switching and routing planes, and are fully consistent with the three-layer hierarchical model. To manage the configuration efficiently and to avoid repeating configuration code, group inheritance was employed by defining a parent group, `[network_devices:children]`. In this way, all subgroups inherit the management properties of the parent group. Code Snippet 4.1 presents the final, operational structure of this inventory file.

Code Snippet 4.1: The inventory file structure (inventory.ini) and the logical grouping of devices by IP

```
[routers]
R1 ansible_host=192.168.10.254

[core_switches]
swDistribution ansible_host=192.168.10.1

[access_switches]
swEdu ansible_host=192.168.10.11
swTech ansible_host=192.168.10.12

[network_devices:children]
routers
core_switches
access_switches

[network_devices:vars]
ansible_user=admin
ansible_password=123
# WARNING: Credentials in plaintext for lab only.
# In production, encrypt using Ansible Vault:
# ansible-vault encrypt_string 'your_password'
ansible_network_os=ios
ansible_connection=network_cli
ansible_become=yes
ansible_become_method=enable
ansible_become_password=
```

4.5.2. Breaking Down the Behavioral Parameters and Secure Sessions in the Automation Layer

The engineering value of this inventory file resides in the global-variables block, [network_devices:vars]. These parameters determine how the SSH sessions behave, authenticate, and are encapsulated:

- `ansible_host`: this key variable maps the device's logical alias (such as R1 or swEdu) directly to its static management IP address. Its use means that the Ansible engine requires neither DNS nor OS-level hostname configuration in order to open SSH sessions; it communicates instead over the fixed, secure IP addresses defined here.
- `ansible_user` and `ansible_password`: these define the devices' local AAA authentication credentials centrally, so that Ansible can use them when opening the SSH tunnel.

- `ansible_network_os=ios`: this explicitly informs the Ansible engine that the target devices run Cisco IOS, which causes Cisco's native command plugins to be loaded when the YAML code is compiled.
- `ansible_connection=network_cli`: unlike Linux servers, which use standard SSH connections, network devices require the `network_cli` plugin. This plugin establishes a persistent SSH session that remains open throughout all tasks, dramatically reducing the overhead of repeatedly opening and closing secure tunnels.
- Privilege escalation (`ansible_become`): because applying settings on Cisco devices requires privileged exec mode, `ansible_become=yes` was enabled. With `ansible_become_method=enable`, Ansible is instructed to issue Cisco's native `enable` command in order to escalate privileges. Leaving `ansible_become_password` empty indicates that no separate enable-mode password was set in the laboratory environment, which accelerates the automated runs.

4.5.3. Logical Grouping and Variable Hierarchy

For efficient management, the network devices in the inventory were partitioned into logical groups. For example, the access-layer switches (`swEdu` and `swTech`) were placed in a group named `[access_switches]`, and the distribution switch in `[core_switches]`. This logical grouping enables the administrator to assign common variables (such as `username`, `password`, and `OS type`) to an entire group through the `group_vars` directory, rather than defining them for each device individually. This substantially reduces repetitive code (the DRY principle: Don't Repeat Yourself) and greatly improves the readability and maintainability of the project.

4.6. Breaking Down the Code Architecture and Implementing the Operational Scenarios

With the control node and the inventory file in place, the infrastructure-development and coding phase commenced. The project's files and directories were arranged in a modular directory structure. All playbooks were organized into a separate directory called `playbooks`, in accordance with the separation-of-concerns principle from software engineering [8]. The implementation strategy was to develop independent, single-responsibility playbooks. On this basis, five distinct scenarios were developed as YAML files:

- Dynamic backup (automated network backup);
- Baseline standardization (configuration standardization);
- VLAN provisioning;
- Security hardening;
- Monitoring preparation.

In all of these scripts, the parameter `gather_facts: false` was set deliberately. Because gathering system facts on network devices is slow and imposes an additional load on the routers and switches, disabling it in scenarios that do not require system variables (such as the device serial number) noticeably optimizes the execution time of the code [1].

4.7. Scenario One: A Dynamic, Automated Backup System (Dynamic Backup Automation)

One of the fundamental challenges in managing enterprise networks is maintaining the latest copy of the device configurations (the running configuration) prior to making any change. The `1_backup.yml` file was developed to address this need and to create an automated archiving system.

4.7.1. Workflow Analysis

Rather than issuing commands indiscriminately, this playbook employs a multi-step logic that relies on the Jinja2 templating engine:

- Timestamping: first, using the `command` module to invoke the Linux `date` command, the precise system date and time are obtained and registered in a variable named `date_var`.
- Local delegation: the key engineering point here is the use of `delegate_to: localhost`. This directs Ansible to create the directory (using the `file` module) not on the Cisco switches but on the Ubuntu server (the control node) itself. This technique is an architectural necessity for hybrid processes that require interaction between the host OS and the network devices [8].
- Extraction and storage: finally, the `ios_command` module executes `show run` on the network devices and stores the output in temporary memory. The `copy` module then saves it to the pre-created path (using Jinja2 mapping such as `{{ date_var.stdout }}`) under each device's

own name ({{ inventory_hostname }}.txt). This cycle eliminates any dependence on commercial backup software such as SolarWinds NCM.

Code Snippet 4.2: The backup playbook

```
---
- name: Scenario 1 - Automated Network Backup
  hosts: network_devices
  gather_facts: false
  tasks:
    - name: Get Date
      command: date +%Y-%m-%d_%H:%M
      register: date_var
      delegate_to: localhost
    - name: Create Directory
      file:
        path: ./backups/{{ date_var.stdout }}
        state: directory
      delegate_to: localhost
    - name: Get Config
      ios_command:
        commands: show run
      register: config_output
    - name: Save to File
      copy:
        content: "{{ config_output.stdout[0] }}"
        dest: "./backups/{{ date_var.stdout }}/{{ inventory_hostname }}.txt"
      delegate_to: localhost
```

4.8. Scenario Two: Implementing a Baseline and Standardization (Baseline Standardization)

Having secured the data with a backup system, the 2_Standardize.yml file is executed in order to harmonize the base settings across the entire access and distribution layers. The objective of this scenario is to ensure that uniform policies are applied to all nodes.

4.8.1. Declarative State Management

This script uses the powerful `ios_config` module. Rather than entering commands at the CLI, this module treats the device's configuration as a state.

- Encryption and DNS: commands such as `service password-encryption` (to prevent passwords from appearing in clear text) and DNS server settings (`ip name-server 192.168.10.10`) are injected into the devices as lines. Using the local DNS server established on Windows Server in Phase One enables the switches to resolve domain names themselves.

- Banner management (MOTD banner): using the dedicated `ios_banner` module, an “Authorized Access Only” warning was configured on all devices. The use of the `|` character in YAML permits multi-line strings while preserving the visual structure, which is necessary for designing standard enterprise banners.

4.8.2. Idempotency in Action

The cornerstone of Ansible’s architecture appears in the final task of this scenario, where the `save_when: modified` parameter is used in the `ios_config` module. In traditional networks, after every change the administrator invariably issues write memory after every change, performing redundant write operations to non-volatile memory even when the configuration has not actually changed [15]. This parameter, by contrast, instructs Ansible to first compare the device’s running configuration with the startup configuration and to save the changes to NVRAM if, and only if, a change was made in the current run. This is the true expression of idempotency in NetDevOps, and it maintains the stability of the network across repeated runs.

Code Snippet 4.3: The security standardization script

```
---
- name: Scenario 2 - Apply Standard Configuration
  hosts: network_devices
  gather_facts: false
  tasks:
    - name: Set login Warning Banner
      ios_banner:
        banner: motd
        text: |
          #####
          #          WARNING: AUTHORIZED ACCESS ONLY          #
          #  All activity on this device is monitored.          #
          #          Disconnect IMMEDIATELY if unsure!          #
          #####
        state: present
    - name: Encrypt Clear-text Passwords
      ios_config:
        lines:
          - service password-encryption
    - name: Configure DNS Server
      ios_config:
        lines:
          - ip name-server 192.168.10.10
          - ip domain-lookup
    - name: Save Configuration to NVRAM
      ios_config:
        save_when: modified
```

4.9. Workflow Integration and the Execution Logic of the Scenario

Hierarchy

Before proceeding to the code architecture of the subsequent scenarios, it is worth explaining the logic and strategy governing the execution of these playbooks. In NetDevOps, the execution of scripts is not a random process; it follows a carefully engineered pipeline. In this project, the scenarios were designed so that the output of each phase constitutes the prerequisite for the next. First, the execution of Scenario One (backup) establishes the device's current state as a safe restore point. Scenario Two then standardizes the base platform and the initial access. Once this foundation is established, Scenario Three (VLAN configuration) shapes the Layer 2 logical architecture. Finally, building upon this, Scenario Four hardens the security of the network, and Scenario Five prepares the infrastructure for connection to the centralized monitoring system (Zabbix) in the forthcoming phases. This logical continuity substantially reduces the risk of configuration drift across all Ansible-managed parameters and emulates a CI/CD approach within the network infrastructure [9].

4.10. Scenario Three: Provisioning Logical Infrastructure and Data-Driven Management of Virtual Networks

The `3_vlan.yml` file is responsible for deploying the network's Layer 2 architecture. The engineering value of this script, however, lies not merely in the creation of a few simple VLANs, but in the paradigm shift from static configuration to data-driven configuration.

4.10.1. Removing VTP and Moving the Source of Truth

In the first task of this playbook, the VTP (VLAN Trunking Protocol) mode on all switches was set to transparent. In traditional networks, VTP is used to distribute VLAN information automatically between switches; the protocol, however, is susceptible to a dangerous vulnerability known as the "VTP bomb," whereby a newly introduced switch with a higher revision number can erase the organization's entire VLAN database [14]. In NetDevOps thinking, such dynamic protocols are regarded as obsolete. By disabling VTP, it was explicitly declared that the network's single source of truth is no longer the devices themselves, but the Ansible code and repository, which determine the network's final state.

4.10.2. Loop Architecture and Separating Data from Logic

In this script, the list of VLANs (Management, Edu, Tech, Native) is defined not within the command body but as a dictionary data structure in the vars section (the my_vlans variable). Then, using the loop directive (loop: "{{ my_vlans }}") and the ios_config module, the information is read dynamically and injected into the switch.

Engineering benefit: with this architecture, should the organization ever need to add dozens of new VLANs, the administrator need not touch or modify the execution logic at all; it suffices to add the new data to the variable list. This decouples the execution logic from the data, enabling the playbook to scale horizontally across an arbitrarily large inventory of devices without any modification to the task code.

Code Snippet 4.4: Ansible playbook for dynamically provisioning VLANs (data-driven architecture)

```
---
- name: Scenario 3 - Provision VLANs
  hosts: core_switches, access_switches
  gather_facts: false
  vars:
    my_vlans:
      - { id: 10, name: mgmt }
      - { id: 40, name: edu }
      - { id: 50, name: tech }
      - { id: 99, name: native }
      - { id: 71, name: Test }
  tasks:
    - name: Set VTP Mode to Transparent
      ios_config:
        lines:
          - vtp mode transparent
    - name: Create and Name VLANs with ios_config
      ios_config:
        parents: vlan {{ item.id }}
        lines:
          - name {{ item.name }}
      loop: "{{ my_vlans }}"
    - name: Save Config
      ios_config:
        save_when: modified
```

4.11. Scenario Four: Security Hardening and Reducing the Attack Surface

The 4_security.yml file is one of the most critical components of the automation, developed to bring the devices into alignment with security standards (such as the CIS Benchmarks). In large networks, configuring security settings manually on each port and switch carries a high risk of

human error and may create latent vulnerabilities. Ansible guarantees that security policies are applied uniformly across the entire network.

4.11.1. Stopping Vulnerable Services and Controlling Discovery Protocols

The default services of Cisco devices (such as the HTTP server and BOOTP) are regarded as threats in modern networks because they do not support strong encryption. In this playbook, these services were disabled using the lines structure. In addition, the Cisco Discovery Protocol (CDP)—which can leak sensitive topology information, OS versions, and IP addresses to attackers connected to access-layer ports (reconnaissance attacks)—was disabled globally with the `no cdp run` command [17].

4.11.2. Securing Remote Access Lines (VTY & Console Lines)

One of the most important steps here is the complete blocking of the insecure Telnet protocol and the enforcement of SSH version 2 (SSHv2) on the devices. This neutralizes the risk of sniffing attacks and credential theft. A noteworthy aspect of the development of this code is its deliberate awareness of the runtime environment: as the comments within the code correctly note, the auto-disconnect time (`exec-timeout`) for the console port was set to `0 0` (unlimited) on account of the project's laboratory nature, in order to avoid frequent disconnections during development. Owing to careful documentation, however, the script is fully prepared for adjustment to industry-standard values (such as `5 0`) for production deployment.

Code Snippet 4.5: The security hardening script

```
---
- name: Scenario 4 - Security Hardening
  hosts: access_switches, core_switches
  gather_facts: false
  tasks:
    - name: Disable Unused Services (HTTP, BOOTP, CDP)
      ios_config:
        lines:
          - no ip http server
          # In real world also add: no ip http secure-server
          - no ip bootp server
          - no cdp run
          - service password-encryption
    - name: Enforce SSH Version 2 and Timeouts
      ios_config:
        lines:
          - ip ssh version 2
          - ip ssh time-out 60
          - ip ssh authentication-retries 3
    - name: Secure VTY Lines (kill Telnet)
      ios_config:
        parents: line vty 0 4
        lines:
          - transport input ssh
          - exec-timeout 5 0
          - logging synchronous
    - name: Configure Console Line for Lab (Never Timeout)
      ios_config:
        parents: line con 0
        lines:
          - logging synchronous
          - exec-timeout 0 0
          # Should be 5 0 in production, 0 0 in lab demo like here
    - name: Save Config
      ios_config:
        save_when: modified
```

4.12. Scenario Five: Instrumentation and Preparing the Monitoring Foundation

The 5_monitoring_prep.yml file is the missing link and the bridge between Phase Two (automation) and Phase Three (monitoring) of this project. A powerful monitoring system such as Zabbix is entirely ineffective without standardized access to the network nodes.

This automation scenario (enabling SNMP on the switches) is, in effect, the cornerstone of Phase Three. The deep connection between this scenario's code and the establishment of the Zabbix observability platform is analyzed in Section 5.6 of Chapter Five, in order to demonstrate how automation constitutes a prerequisite for monitoring.

4.12.1. Unified Configuration of SNMP

In this scenario, SNMP version 2c was enabled on all access- and distribution-layer devices. The critical variables—including the community string named `snmp_pass`—were configured with restricted, read-only (RO) access, so that even if the community string were exposed, an attacker could not alter the infrastructure settings [24].

The benefit of performing this with Ansible: SNMP settings are the component most prone to human error (such as a typographical error in the monitoring server’s address or the community string), which gives rise to “dark spots” in the organization’s monitoring dashboards. By using Ansible and setting `zabbixServer_ip: 192.168.10.20` as a global variable, it was ensured that all switches connect to the Ubuntu server with an identical structure—without the slightest deviation—and are prepared to transmit traffic data, port status, and CPU usage to the Zabbix server in the next phase.

The `snmp-server` host directive additionally registers the Zabbix server as an SNMP trap destination, complementing the poll-based collection (`GetRequest`) used as the primary mechanism in Chapter Five.

Code Snippet 4.6: SNMP configuration script for integration with the monitoring system

```
---
- name: Phase 3 - Step 1 Enable SNMP on Devices
  hosts: network_devices
  gather_facts: false
  vars:
    snmp_pass: "public"
    # WARNING: This value is for lab/demo purposes only.
    # In production, use Ansible Vault to encrypt sensitive credentials.
    # Reference: https://docs.ansible.com/ansible/latest/vault_guide/index.html
    zabbixServer_ip: "192.168.10.20"
  tasks:
    - name: Configure SNMP Server
      ios_config:
        lines:
          - snmp-server community {{ snmp_pass }} RO
          - snmp-server host {{ zabbixServer_ip }} version 2c {{ snmp_pass }}
    - name: Save Config
      ios_config:
        save_when: modified
```

The community string was intentionally left at its conventional default value (`public`) in this laboratory environment to keep the demonstration clear and reproducible; because the access

level is restricted to read-only (RO), an exposed string cannot be used to modify any device configuration, and—as detailed in Section 6.6.2—a source-address ACL further restricts SNMP access exclusively to the Zabbix server. In a production deployment this value would be replaced with a non-default, secret community string.

4.13. Analyzing the Execution Output and Validating the Desired State

The development of automation scripts constitutes only half of NetDevOps; the other, critical half is their correct execution and the analysis of the system’s feedback. Once the five playbooks were complete, configuration deployment was initiated using the `ansible-playbook` command from the command line of the Ubuntu server.

4.13.1. Play Recap Analysis

When playbooks are executed against the network, Ansible’s engine provides a summary report, the Play Recap, after processing each node. On the first execution of these scenarios, all switches were reported with a changed status. This indicated that the current state of the devices did not match the desired state defined in the code, and Ansible therefore injected the commands into the devices’ memory [8].

The most important technical outcome of this phase emerged on the second execution. When the scenarios were run again (with no change to the code), the engine—after inspecting the devices’ state—reported them as ok (rendered green in the terminal) with `changed=0`. This output is concrete, practical evidence that idempotency was successfully implemented in this project’s network infrastructure: the system attained complete stability, and repeated execution of the code imposes no additional processing load and causes no destructive change.

4.13.2. Transport-Layer Error Handling

During execution, given the processing limitations of the GNS3 emulator, the default SSH timeout parameters required tuning. If Ansible does not receive a response within the standard window, it halts the process with a timeout error. To overcome this environmental challenge, the network environment variables in the Ansible core configuration file (`ansible.cfg`) were tuned, and the `[persistent_connection]` parameter was configured with a longer response time so that the concurrent connections to the switches would not be dropped under heavy load.

Chapter Five: Phase Three – Centralized Monitoring Architecture, Performance Management, and Building Observability

5.1. Chapter Introduction

The deployment of a stable network and its automation with Infrastructure as Code tools remain incomplete without a dynamic monitoring and observability system. In practice, administrators require a tool that examines the hardware and software health of the devices in real time and raises appropriate alerts before critical failures result in downtime. In the preceding phases, the network was designed and the security processes and baselines were deployed with Ansible. This chapter covers Phase Three of the project—establishing the centralized Zabbix monitoring platform on Docker containers and tuning it to collect and analyze critical network data.

5.2. Comparative Evaluation of Infrastructure Monitoring Platforms

Selecting a monitoring platform for a heterogeneous enterprise network (comprising both Cisco devices and Linux/Windows servers) requires a careful examination of the technical criteria. To justify the selection of Zabbix from an engineering standpoint, four well-known industry solutions were analyzed:

- Prometheus: built upon a time-series database (TSDB), it is well suited to cloud-native and containerized environments. Prometheus, however, uses a pull model over HTTP, whereas traditional network devices (such as Cisco switches) lack native Prometheus exporters. Its use would necessitate the deployment of complex middleware (such as the SNMP Exporter), which increases management overhead [19].
- Nagios: one of the oldest open-source tools, it performs monitoring on the basis of periodic scripts (plugins). Its principal weakness is its traditional, file-based architecture. Modifying settings requires the manual editing of large text files and the restarting of the service, which is at odds with the agility demanded by NetDevOps [20].
- PRTG Network Monitor: a highly capable proprietary, commercial tool for network monitoring on Windows. Although PRTG offers an attractive user interface, it was excluded

from consideration in this project because it is closed-source, its free edition is severely limited (capped at 100 sensors), and it cannot be integrated deeply with Linux automation pipelines.

- Zabbix: an enterprise-grade, fully open-source platform, selected as the definitive monitoring solution for this project for the following reasons:
- Native SNMP support: it can interact directly with the Cisco IOS core without any middleware.
- Modular architecture: the complete separation of the processing layer (Zabbix Server), the presentation layer (Zabbix web), and the storage layer (the database).
- Dynamism and automation capability: highly capable JSON-RPC-based APIs that also enable Ansible to automate the registration of nodes in the monitoring system.

5.3. The Containerization Paradigm: Rationale for Migrating to Docker

Having selected Zabbix, the manner of its deployment constituted the next architectural challenge. Installing software in the traditional manner, natively on the operating system (on bare metal or on traditional VMs), leads to infrastructure rigidity and to dependency conflicts (“dependency hell”) [22]. To address this, Docker containerization was selected as the principal deployment platform.

5.3.1. What Is Docker, and How Does It Differ from Traditional Virtualization?

In traditional virtualization (such as the hypervisor structure in VMware), each virtual machine requires a complete guest operating system together with its own dedicated kernel, which consumes substantial memory and CPU resources. Docker, by contrast, is an open-source containerization technology that allows applications to run within fully isolated environments called containers. Unlike virtual machines, containers share the host operating system’s kernel and employ Linux kernel features such as namespaces (for isolating resources) and control groups (cgroups, for limiting hardware-resource usage) [22]. This renders containers extremely lightweight and reduces startup time from several minutes (for full virtual machines) to a few seconds.

5.3.2. The Benefits of Deploying Zabbix on Docker in This Project

The use of Docker to establish Zabbix in this project conferred the following strategic benefits:

- Environment portability: Zabbix requires a web server (Apache/Nginx), a PHP interpreter with extensive PHP dependencies and ancillary libraries, and a database (MySQL/PostgreSQL) in order to operate. Through containerization, this entire software stack is deployed as pre-tested containers and behaves identically in the laboratory and in production.
- Ease of life-cycle management: subsequently upgrading the monitoring system reduces to changing the image tag—accomplished in seconds and without harm to the existing data.

5.4. Local Deployment Challenges: Circumventing International Sanctions and Migrating to Domestic Mirrors

During the deployment of Docker on the Ubuntu server (192.168.10.20), the project encountered one of the most substantial and demanding operational challenges in domestic network infrastructure: the restrictions arising from international sanctions and reciprocal filtering.

5.4.1. Technical Analysis of the Docker Hub Blockage

The official Docker Hub server (registry.docker.io)—the principal source for downloading standard images—blocks requests originating from Iranian IP blocks in accordance with sanctions policies, returning the well-known HTTP 403 Forbidden error or a TLS handshake timeout. This entirely halted the download of the critical Zabbix images.

Code Snippet 5.1: A sample transport-layer error when calling Docker Hub

```
Error response from daemon: Get "https://registry-1.docker.io/v2/":
net/http: TLS handshake timeout
```

5.4.2. The Engineering Solution: Configuring Docker Registry Mirrors

To overcome this structural challenge and to continue the deployment without relying on unstable traffic-encryption tools (VPNs, which themselves caused a sharp reduction in bandwidth and instability within the hypervisor environment), the Docker image-pull infrastructure was re-engineered. Docker on the Ubuntu server was configured to use domestic mirror and intermediary servers (Iranian local mirrors) instead of pointing to the original servers. This was accomplished by editing and applying native settings in the Docker daemon

configuration file (`/etc/docker/daemon.json`) [23]. By adding the addresses of stable domestic mirrors—such as ArvanCloud or Hamravesht—to the `registry-mirrors` array, the container-download requests were routed first to these domestic intermediary servers (which act as a centralized cache), so that the Zabbix images were downloaded and deployed at the full speed of the physical line. This constitutes a clear example of managing an infrastructure crisis under technological sanctions.

5.5. Data Infrastructure Architecture and Ensuring Data Persistence

One of the principal pillars in deploying enterprise monitoring platforms is the design of the data-storage layer. Zabbix continuously collects a vast volume of time-series data—including traffic metrics, hardware status, and system logs.

5.5.1. Choosing and Examining the Database in the Container Environment

In this project, a relational database (RDBMS) container was deployed as Zabbix’s principal storage engine. In Docker’s architecture, if a database container is established without special measures, the data is stored temporarily in the container’s writable layer—meaning that when the container is stopped or deleted, all historical monitoring data (History and Trends) is erased entirely [22].

5.5.2. Implementing Docker Volumes for Data Persistence

To overcome the problem of data ephemerality, the Docker Volumes mechanism was employed. In the `docker-compose.yml` configuration file, a physical directory of the host operating system (Ubuntu Server) was mounted to the database container’s internal directory (`/var/lib/postgresql/data`). This decoupling of computation from storage guarantees that, even if the database container fails or is upgraded, the data structure and the network tables remain completely intact. Environment variables, including credentials, were defined in the Compose file. For laboratory purposes, these are stored in plaintext; in production environments, Docker Secrets or an external vault should be used to protect sensitive values.

5.6. Analyzing the Structural Link Between the Automation and Monitoring Phases

The relationship between Phase Two (automation) and Phase Three (monitoring) is one of cause and effect, and it is synergistic. To collect data from the access- and distribution-layer devices, Zabbix depends entirely on SNMP.

5.6.1. The Role of Ansible's Fifth Scenario in Enabling Monitoring

Without the execution of `5_monitoring_prep.yml` in the automation phase, centralized network monitoring would have been impossible. In that phase, Ansible served as an infrastructure catalyst. The execution of that playbook automatically and simultaneously applied the following to all switches:

- defining a read-only community string (snmp-server community public RO);
- configuring the SNMP notification destination to point exclusively to the Zabbix server's IP address (192.168.10.20) using the snmp-server host directive.

5.6.2. The Engineering Benefit of Eliminating Configuration Drift

Had this been performed traditionally and manually, a single typographical error in the community string or in the monitoring server's IP address on any one switch would have created a network blind spot. Ansible's idempotent execution guaranteed a uniform communication baseline across all nodes, eliminating the device-by-device inconsistencies inherent in manual CLI configuration. As a result, as soon as the containers came up, Zabbix was able to begin polling immediately—without requiring any local reconfiguration on the switches [8].

5.7. Engineering the Centralized Dashboard and Analyzing Live Metrics

The Zabbix dashboard serves as the command-and-control center for the entire infrastructure of this project. This web interface transforms the raw data received from the network protocols into comprehensible visual layers.

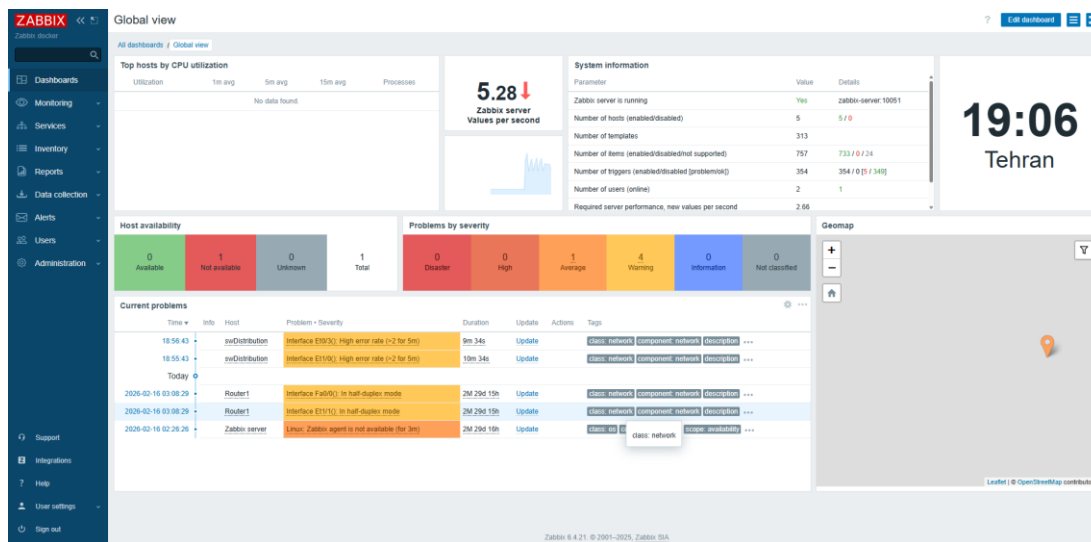


Figure 5.1. The centralized Zabbix monitoring dashboard and the stability status of the network nodes

The dashboard was configured to present the administrator with the following at a glance:

- a device-health status table (System Status) based on standard color codes (green for normal, yellow for warning, red for critical);
- the rate of events (System Top Triggers), to identify the nodes bearing the greatest traffic load or the most errors;
- a live availability chart for the devices based on the ICMP and SNMP protocols.

5.8. Examining the Telemetry Transport Layer: Why SNMP?

To collect critical metrics from network hardware, a standard, lightweight, and universal application-layer protocol is required. SNMP (Simple Network Management Protocol) was selected as the industry's gold standard for this purpose [24].

5.8.1. SNMP's Information Architecture: The MIB and OID Concepts

Understanding how Zabbix operates in this phase requires an explanation of the internal architecture of the network devices' database. Every Cisco device stores its information in a hierarchical tree structure called the MIB (Management Information Base). Each point, or leaf, in this tree has a unique numeric address called an OID (Object Identifier) [25]. For example, CPU usage over a given window, or the traffic status of a specific port, each possesses its own dedicated OID (such as 1.3.6.1.2.1.2.2.1.10).

The Zabbix server (the SNMP manager) periodically transmits its requests as SNMP GetRequest packets over the unreliable but fast UDP protocol (port 161) to the switches (the SNMP agents); the switch then extracts the data from its MIB and returns a response. This polling process maintains the continuity of monitoring.

SNMP Architecture

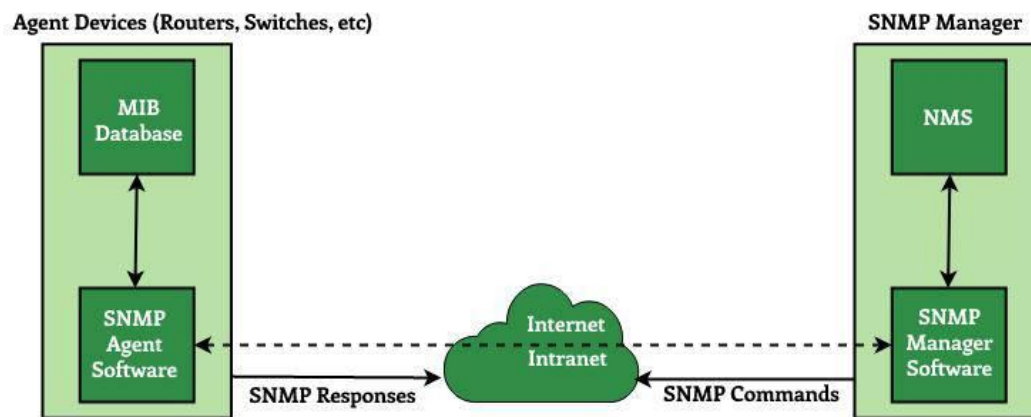


Figure 5.2. High-level communication architecture of SNMP and the data flow between the manager and the agent [26]

5.9. The Operational Monitoring Subsystem and Node Availability Analysis

After Ansible had injected the baselines, the interactive foundation was established at the transport layer. At this stage, the network nodes were registered in Zabbix's monitoring menu under Monitoring → Hosts. This section, as the live-status layer, is responsible for continuously monitoring and instantaneously displaying the availability of the devices.

5.9.1. Analyzing the Availability Indicator Mechanism

At this layer, Zabbix indicates each device's connectivity status through color codes in the availability matrix (the SNMP section).

- **Green indicator (operational state):** indicates that the monitoring server has full access to the MIB database of the switch or router. Here, SNMP GetRequest packets are transmitted at the defined intervals and the corresponding responses are returned without packet loss.

- Red indicator (failure state): indicates a break in the communication path or at the service layer. The indicator turning red shows that the “Host Unreachable” trigger has fired, which occurs when the reply packets are not received within the permitted timeout window. This subsystem constitutes the principal basis for the network’s statistical availability (SLA) analysis [18].

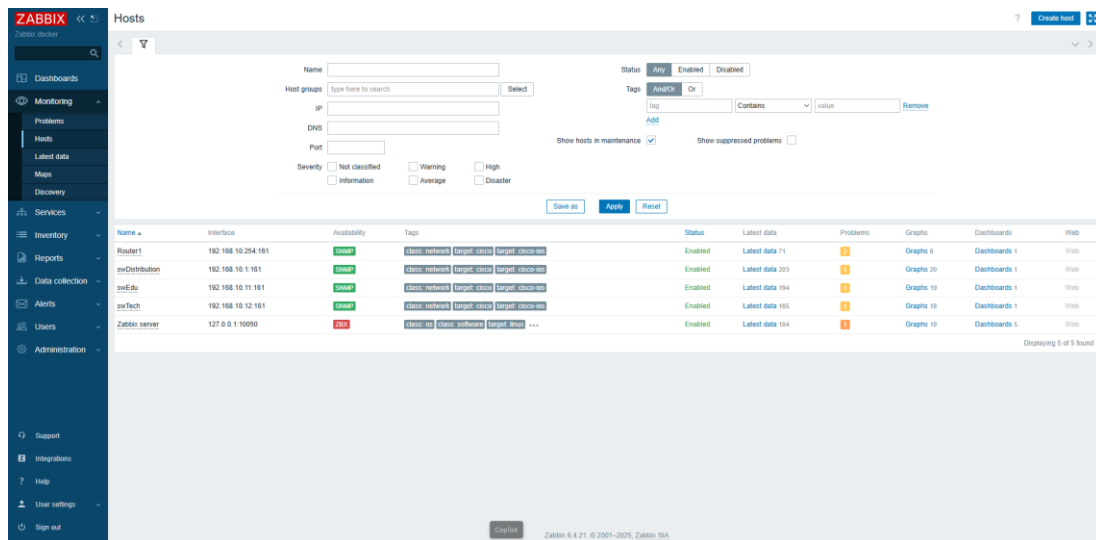


Figure 5.3. Configuration of the network devices’ inventory database in Zabbix

5.10. Dynamic Monitoring with Templates and the Magic of LLD

Monitoring every individual port and thermal sensor on routers and switches manually is impractical and error-prone. If a switch has 48 ports, monitoring the error rate, inbound traffic, outbound traffic, and disconnection status of each port entails defining more than 200 separate items for a single switch alone. Zabbix’s architectural answer to this is the use of templates and the Low-Level Discovery (LLD) capability.

5.10.1. The Concept of Inheritance in Zabbix Templates

A template is a pre-built collection of items, triggers, graphs, and dashboards that is attached to one or more hosts at once. In this project, the standard, native “Cisco IOS by SNMP” template was used. Attaching this template to the access and distribution switches activates the inheritance principle from software engineering, whereby the switch inherits all of Cisco’s specialized monitoring capabilities [18].

5.10.2. Examining the Low-Level Discovery (LLD) Mechanism

The principal technical advantage of Zabbix templates is their reliance on LLD. Rather than monitoring the switch's ports in a hard-coded manner, LLD first performs a structural scan of the switch's MIB tree to determine exactly which interfaces the particular switch possesses (such as Ethernet0/0 or Ethernet0/1, as used in this project's IOL-based switches). Having dynamically discovered the interfaces, Zabbix automatically creates monitoring slots for each active or inactive port. This intelligent process guarantees that, should a new port be added to the switch in future, or its modular layout change, the monitoring system adapts to the new arrangement on its own—without the intervention of a technician—and updates the telemetry accordingly [21].

5.11. Examining the Core/Edge Router's Operational Parameters (Router 1 Live Parameters Analysis)

Because R1 plays a strategic role in connecting the internal network to the internet (the NAT environment in VMnet8), monitoring it closely is of considerable importance. After the standard “Cisco IOS by SNMP” template was attached to this node, Zabbix began to extract the following critical parameters:

5.11.1. Breaking Down R1's Monitored Metrics

- CPU and memory utilization: expressed as a percentage, this immediately reveals any denial-of-service (DoS) attack or memory leak in the router's operating system.
- Interface traffic (bits in/out): using the OIDs in the IF-MIB, the bandwidth consumed on the link to the internet and on the link to the distribution switch is monitored as flow charts.
- Interface operational status: if a link is physically severed or shut down in software, Zabbix immediately raises a High-severity trigger.
- ICMP ping latency and packet loss: this measures the quality of the outbound link and the stability of the internet connection within the scenario.

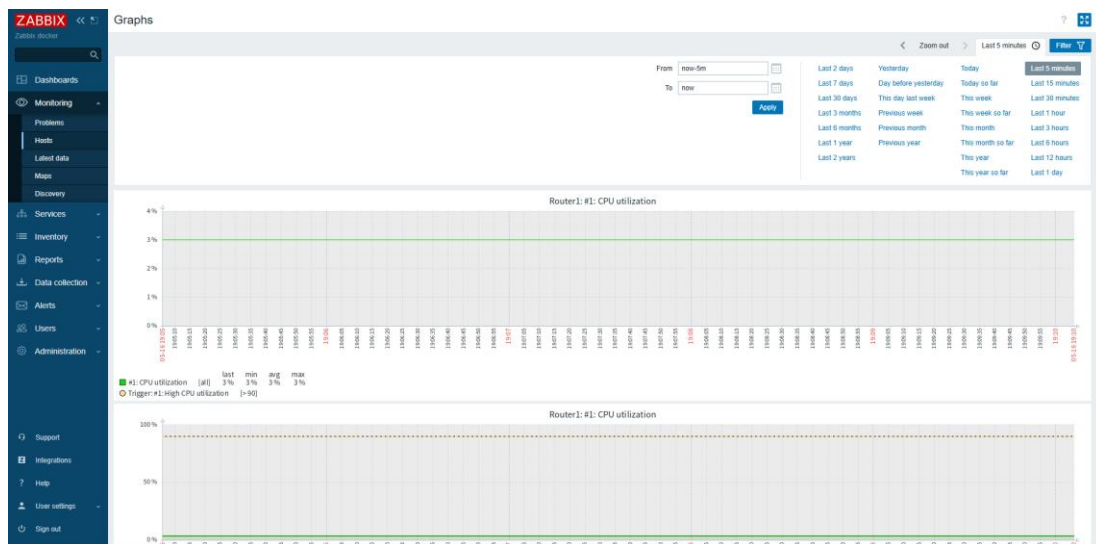


Figure 5.4. Live charts of interface traffic and hardware-resource usage on router R1

5.12. The Intelligent Event Analysis Layer: Trigger Architecture and Alert Thresholds

The mere collection of network data and metrics through SNMP has no operational value in the absence of a logical analysis layer. Zabbix performs this intelligent evaluation by means of a construct called a trigger. Triggers are, in essence, logical and mathematical expressions that evaluate the state of a metric and determine whether the network is operating normally or is in a critical condition [21].

5.12.1. Explaining the Logic of the Conditional Expressions and Math Functions in Triggers

Each trigger follows a formula, or conditional expression, applied to the data received from the items. In this project, advanced functions such as `last()` (to check the most recent value), `max()` (to assess the peak usage over a window), and `nodata()` (to detect a complete loss of the telemetry stream) were used to monitor the infrastructure.

For example, to monitor R1's CPU usage, a trigger was defined with the following logic: if the average CPU usage over the preceding five minutes exceeds 85%, the trigger's state changes from OK to Problem. This approach prevents false alarms arising from momentary CPU spikes and maintains the stability of the monitoring system [27].

5.12.2. The Alert Flapping Problem and Applying Hysteresis

One of the major challenges in enterprise networks is flapping, whereby a trigger oscillates between OK and Problem from one moment to the next. This occurs when a metric (such as bandwidth or device temperature) hovers precisely at the threshold (say, 80%). It gives rise to alarm fatigue among the network support team. To address this, the mathematical concept of hysteresis was applied in configuring this project's sensitive triggers. Under this method, the condition for entering the critical state differs from the condition for leaving it: for example, the trigger activates when RAM usage reaches 90%, but deactivates and clears only when usage falls below 80%. This 10% gap confers exceptional stability on the monitoring system [28].

5.13. Classifying Event Severity Levels (Severity Levels Mapping)

Not all network events carry equal weight. A trunk link failing between the distribution and core switches threatens the stability of the entire organization, whereas the filling of an access switch's buffer is of far lesser consequence. Zabbix permits the classification of events into six severity levels, which in this project were localized and implemented as follows:

Table 5.1. Event severity levels, standard color codes, and their operational meaning in the Zabbix monitoring system

Severity	Standard Color	Operational Meaning in the Project Topology
Not Classified	Gray	Unknown events or unclassified logs
Information	Light blue	Minor changes, such as a config change without service interruption
Warning	Yellow	An interface's traffic rising above 70% of link capacity
Average	Orange	Rising CPU or RAM usage on access-layer devices
High	Light red	A secondary link or a server-connection port going down
Disaster	Dark red	The edge router (R1) or distribution switch going down / a database error

This hierarchical structure enables network technicians, during concurrent crises, to devote their effort first to resolving Disaster- and High-level errors and to optimize the use of their resources [28].

5.14. Alerting Mechanisms and the Escalation Process

The final stage of the monitoring cycle is the intelligent notification of network administrators regarding events. Beyond merely displaying the error in the web interface, Zabbix employs the Actions subsystem to dispatch commands and ancillary alerts.

5.14.1. Escalation Path Architecture and Alerting Design

In this project, the Zabbix alerting architecture was configured to display triggered events directly on the centralized dashboard with color-coded severity indicators, affording the network administrator immediate visual awareness of any infrastructure issue. The platform's native escalation capability—which supports multi-step notification workflows, including the dispatch of alerts to shift technicians, escalation to management after a defined timeout, and the issuing of remote remediation commands—was analyzed and documented as a structured development path for the production phase. In a production environment, this escalation chain would be activated by configuring a notification media type (such as SMTP email or a messaging webhook) and enabling the corresponding trigger action [21].

5.14.2. The Interactive Notification Platform in Domestic Infrastructure

Although Zabbix uses SMTP to send email by default, in the operational implementation phase of this project—given how frequently email is overlooked in work settings—the possibility of connecting Zabbix to the webhooks of centralized messaging platforms and notification bots through HTTP POST and JSON data structures was explored. This integration drives the organization's error-response time (Mean Time to Repair, MTTR) down to the lowest possible value.

5.15. Examining the Container Orchestration: Deploying Zabbix on Docker

As noted at the beginning of this chapter, the monitoring system was deployed on Docker. Running several interrelated services simultaneously (database, processing server, web

interface), however, requires a tool for orchestration. Docker Compose was used for this purpose.

5.15.1. Analyzing the Docker-Compose File Structure and Defining the Services

Using a declarative YAML file, all of the infrastructure requirements of the Zabbix platform were modeled. This approach extends the Infrastructure as Code concept from the management of network devices (Ansible) to the management of the monitoring servers themselves.

Code Snippet 5.2: The Zabbix container orchestration configuration in a Compose file

```
version: '3.5'
services:
  postgres-server:
    image: postgres:13-alpine
    environment:
      - POSTGRES_USER=zabbix
      - POSTGRES_PASSWORD=zabbix_pwd
      - POSTGRES_DB=zabbix
    volumes:
      - zabbix_db_data:/var/lib/postgresql/data
    restart: always
  zabbix-server:
    image: zabbix/zabbix-server-pgsql:ubuntu-6.4-latest
    ports:
      - "10051:10051"
    environment:
      - DB_SERVER_HOST=postgres-server
      - POSTGRES_USER=zabbix
      - POSTGRES_PASSWORD=zabbix_pwd
      - POSTGRES_DB=zabbix
    depends_on:
      - postgres-server
    restart: always
  zabbix-web:
    image: zabbix/zabbix-web-nginx-pgsql:ubuntu-6.4-latest
    ports:
      - "8080:8080"
    environment:
      - ZBX_SERVER_HOST=zabbix-server
      - DB_SERVER_HOST=postgres-server
      - POSTGRES_USER=zabbix
      - POSTGRES_PASSWORD=zabbix_pwd
      - POSTGRES_DB=zabbix
      - PHP_TZ=Asia/Tehran
    depends_on:
      - zabbix-server
    restart: always
volumes:
  zabbix_db_data:
```

5.15.2. Technical Analysis of the Deployment Parameters

In the snippet above, the following critical mechanisms were implemented, upon which the stability of the entire network depends:

- Dependency management: the use of `depends_on` guarantees that the containers start in a logical order. The Zabbix server remains in a waiting state until the database container (`postgres-server`) is fully operational and ready to serve. This prevents a “Database Connection Refused” error at the moment the server starts.
- Bridge networking: by using Docker Compose’s default bridge network, the containers communicate through their service names (such as `DB_SERVER_HOST=postgres-server`) and require no statically assigned internal IP addresses. This substantially improves the scalability of the software [30].
- Time-zone synchronization: the `PHP_TZ=Asia/Tehran` parameter in the web container ensures that all charts, triggers, and network logs correspond precisely to the organization’s local time—which is of considerable importance for troubleshooting and for aligning network events with local time.

5.15.3. The Port Forwarding Architecture and Accessing the Zabbix UI

One of the key benefits of deploying the monitoring platform on Docker is that it obviates the need to assign a separate IP address to the containers within the organization’s Layer 2 network. The containers run within an isolated internal network (the Docker bridge) and receive dynamic addresses.

In this project, host port mapping was used to access the Zabbix management dashboard. Specifically, port 8080 of the Zabbix web container was mapped to port 8080 on the physical address of the Ubuntu server (the Ansible controller). As a result, the administrator need not engage with the container subnets at all; it suffices to enter the address of the main server and the relevant port (`http://192.168.10.20:8080`) in a browser in order to reach the Zabbix interface. This strategy substantially simplifies network address management.

5.16. Visual Monitoring and Designing Interactive Network Maps (Zabbix Network Maps)

Lists and tables are well suited to detailed analysis, but senior network administrators require visualizations in order to grasp the overall state of the infrastructure rapidly. Zabbix offers a powerful tool called Maps, which permits the network's physical and logical topology to be drawn on a graphical canvas.

5.16.1. Drawing the Topology and Binding It to Data Elements

In this phase, a map corresponding to the topology constructed in GNS3 (including router R1, the distribution switch, and the Edu and Tech access switches) was drawn in the Zabbix panel. The real value of this map is that it is not static. Each icon on it is linked directly to its corresponding host. If the swEdu access switch goes offline, the color of its icon on the map immediately changes from green to red, and the corresponding error is displayed beside it.

5.16.2. Dynamic Link Indicators

On the links between the routers and switches on the map, Zabbix 6 expression macros were configured to display live bandwidth data directly on each link. Using the `{?expression}` syntax, the inbound and outbound traffic counters (the `ifHCInOctets` and `ifHCOctets` OIDs) were bound to the link labels—for instance, the swEdu link was labeled In: `{?last(/swEdu/net.if.in[ifHCInOctets.1])}` and Out: `{?last(/swEdu/net.if.out[ifHCOctets.1])}`. This transformed the static network map into a live operational dashboard, rendering traffic bottlenecks immediately visible without the need to open any submenu [18].

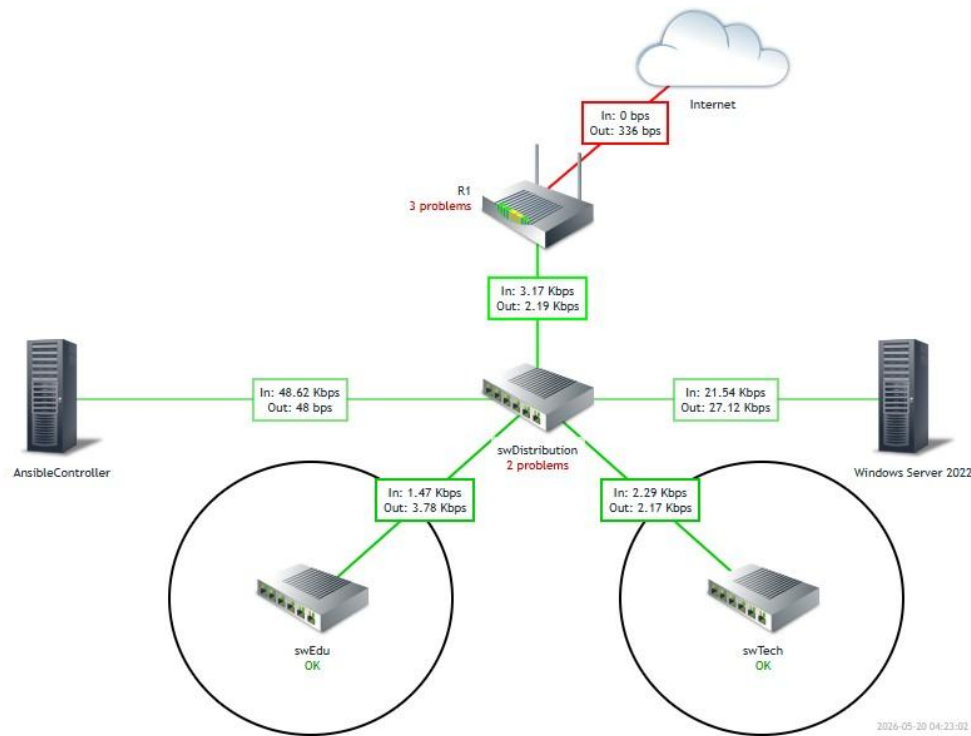


Figure 5.5. The dynamic, interactive Zabbix network map with live key performance indicators for continuous monitoring of the organization's network nodes

5.17. Big-Data Management Strategy and the Storage Life Cycle

One of the hidden but exceedingly critical matters in maintaining monitoring systems is managing the size of the database. The receipt of hundreds of parameters per minute from the Cisco devices can fill the entire storage of the Linux (Ubuntu) server within a few weeks and lead to a system crash.

5.17.1. Separating the History and Trends Architecture

To address the problem of data explosion, Zabbix's architecture divides the collected data into two distinct categories, which were carefully tuned in this project:

- Raw data (History): the precise, instantaneous values received from the devices (for example, the exact CPU usage at 12:01:45). Retaining this data requires considerable disk space. In this network, the retention period for raw data was set to 90 days, balancing short-term diagnostic precision against manageable database growth.

- Processed data (Trends): after a few days, Zabbix averages the raw data hourly (Average), together with the maximum and minimum, and converts it into Trends data. This data occupies very little space. Its retention period was set to 365 days (one year), so that long-term network-growth charts can be produced (capacity planning) and reported to management [31].

This architectural separation preserves the ability to analyze recent network events precisely while preventing the PostgreSQL database within the Docker container from growing exponentially and uncontrollably [33].

5.18. A Scalable Expansion Strategy and Distributed Monitoring Architecture

The design of IT infrastructure should always be founded on the principle of scalability. Although the central Zabbix server can, in the current topology, handle the telemetry stream and receive data from the network's switches and router on its own, in future enterprise-expansion scenarios this centralized architecture will encounter a performance bottleneck [18].

5.18.1. The Challenges of Centralized Monitoring Architecture

When the number of network nodes (access-layer switches, servers, and firewalls) exceeds the thousand-device mark, the central Zabbix server must open tens of thousands of TCP/UDP connections per second in order to collect data. This saturates the server's bandwidth, increases CPU-queue delay, and ultimately results in dropped monitoring packets. Moreover, if the organization comprises distinct physical branches, opening SNMP ports on the edge firewalls for every individual device is a high-risk measure from a security standpoint.

5.18.2. The Conceptual Implementation of the Zabbix Proxy as a Middle-Layer Solution

The engineering solution offered in Zabbix's high-level architecture for this problem is the implementation of distributed monitoring using a Zabbix Proxy. The Zabbix Proxy is a very lightweight Linux daemon that functions as a data collector. It is deployed in the various network segments or physical branches, collects SNMP metrics from the surrounding access-layer

devices, stores and compresses them in a small local database (such as SQLite), and finally transmits all the data over a single secure connection to the central Zabbix server [32].

In this project, the network architecture was designed such that, in future expansion phases, a Zabbix Proxy container can readily be added to the docker-compose file in order to relieve the load on the main server container.

5.19. Critical Analysis and Summary of the Monitoring Phase's Achievements

The successful completion of Phase Three rounded out the infrastructure-management life cycle. Combining the NetDevOps architecture (implemented with Ansible in Phase Two) with the observability platform (Zabbix on Docker) created a self-aware network ecosystem. In this phase, the opaque data flowing at Layers 2 and 3 of the network was transformed into visual dashboards, intelligent triggers, and time-series charts.

Even so, a rigorous engineering evaluation requires consideration of the limitations of the operational environment. In the final assessment, although traffic and hardware-resource monitoring was conducted with high precision, the structural limitations and bugs of the GNS3 emulator in stably running gateway-redundancy protocols (such as HSRP and VRRP) meant that the failover state transitions could not be monitored in real time on this laboratory platform. For this reason, monitoring focused exclusively on closely observing the single-path nodes, the access-layer devices, and the edge router. This challenge demonstrates that the behavior of network redundancy protocols in virtual environments may differ from that on physical equipment (ASIC-based hardware), and that monitoring them requires deployment on physical (bare-metal) hardware in future research.

Ultimately, the use of a containerized approach for the monitoring platform, together with the reproduction of the automation scripts, demonstrated that the management of networks built on single-vendor equipment (such as the Cisco IOS operating system) can move beyond the traditional, CLI-centric mode and be elevated to a modern, software-defined paradigm.

Chapter Six: Conclusion, Technical Evaluation, and Future Development Horizons

6.1. Chapter Introduction

Every engineering project in the fields of infrastructure and information technology, having passed through the stages of design, automation, and the deployment of monitoring, requires a high-level evaluation and critical analysis. This chapter is devoted to the final summary of what was achieved through the implementation of the modern NetDevOps platform in this project. Here, the results of the three project phases are first matched against the original objectives in an evaluation matrix; the engineering value and innovations of the design are then explained; and finally, through an analysis of the existing limitations, a precise roadmap is drawn for the further development of this infrastructure in large organizations.

6.2. A Matrix-Based, Step-by-Step Evaluation of the Three Project Phases

To assess the success of the project, the performance of the infrastructure is examined across three layers: physical/logical (design), orchestration (Ansible), and telemetry (Zabbix). Table 6.1 summarizes how the engineering objectives corresponded to the concrete operational results.

Table 6.1. Strategic objectives matched against the operational achievements of the three project phases

Phase	Objectives Set During Design	Concrete Results	KPI
Phase One: Infrastructure Design	Build a stable hierarchical structure, separate layer traffic, and manage addressing with VLANs.	Three-layer topology (Cisco Top-Down), optimal distribution/access switch config, full Layer 3 SVIs.	Achieved — stable hierarchical topology verified through end-to-end connectivity tests.
Phase Two: Automation with Ansible	Eliminate manual config, speed up changes, harden security, enable base services.	Agentless automation pipeline, uniform config code across vendors, configuration drift eliminated for all Ansible-managed parameters.	Achieved — minor limitations in specific GNS3 IOL module compatibility.
Phase Three: Centralized Monitoring	Full observability, live bandwidth monitoring, proactive alerting, containerized services.	Zabbix 6 on Docker, telemetry over SNMP, proactive alerting via dashboard triggers.	Achieved — all four devices polled successfully via SNMP with proactive alerting active.

6.2.1. Analyzing Phase One's Performance: Reinforcing the Network Backbone

In Phase One, Cisco's standard top-down design model (the Cisco hierarchical model) was selected as the blueprint. The results showed that separating the broadcast domains through subnets and trunking the links on the basis of 802.1Q conferred excellent stability on the network. In this phase, any unwanted Layer 2 traffic was contained, and the processing load on the distribution switch (swDistribution) was kept as optimal as possible [4].

Beyond the Layer 2 foundation, the routing layer was future-proofed by replacing manually written static routes with the open, vendor-neutral OSPF protocol: a single-area (area 0) design advertised all internal subnets dynamically, while the passive-interface command on the user-facing SVIs hardened the control plane against rogue-adjacency (OSPF spoofing) attacks. This dynamic foundation, combined with centralized default-route origination on the edge router, produced a scalable and self-consistent Layer 3 backbone that requires no per-device reconfiguration as the network grows [5], [17].

6.2.2. Analyzing Phase Two's Performance: Leaping to the Infrastructure as Code (IaC) Paradigm

The greatest challenge in managing traditional networks is the reliance on the CLI and the human errors that accompany it. In Phase Two, with the introduction of Ansible, the time required to deploy network baselines (such as VLAN provisioning, security hardening, and SNMP configuration) was reduced from hours to seconds. The safe execution of the playbooks demonstrated that the network's management intelligence can be encoded in structured text files (YAML) and placed under version control [1].

6.2.3. Analyzing Phase Three's Performance: Completing the Network Life Cycle with Observability

An automated network without a monitoring system is comparable to a fast vehicle lacking a dashboard and instruments. In Phase Three, through the establishment of the Zabbix containers, visibility was introduced into the entire infrastructure. The dynamic extraction of properties through LLD made it possible for the administrator to ascertain the live status of each individual port, the CPU usage, and the bandwidth in use—without manual inspection. Combined with the persistent database established for the Docker database, this fully ensured the security of the network's statistical data [21].

6.3. Analyzing the Engineering Achievements and Innovations

This project is more than a simple laboratory implementation; it embodies several high-level engineering achievements and innovations that establish its scientific and practical value for organizations:

- Localizing the development process under infrastructure constraints: one of the operational innovations was the circumvention of the severe international sanctions on Docker Hub in Phase Three. Re-engineering the Docker registry directory and routing the container traffic to domestic mirrors provided a successful model for the stable deployment of services on domestic networks.
- Merging two distinct worlds (networking and DevOps—that is, NetDevOps): this design successfully introduced agile software-engineering and DevOps methodologies (containerization, declarative code, service isolation) into the hardware-based, vendor-

centric world of networking (Cisco equipment). This convergence prepares the infrastructure for entry into the era of software-defined networking (SDN) [16].

- Designing a self-healing-ready escalation system: the architecture of the alerting system and the Zabbix triggers, together with the capability to execute remote commands, laid the foundation for constructing a self-healing network—one in which the system can undertake the initial troubleshooting steps without the direct intervention of a technician.

6.4. Analyzing the Geopolitical Challenges, Sanctions, and Barriers to Deploying Open-Source Infrastructure

The deployment of modern network-management and DevOps systems in operational environments within the country is invariably affected by external challenges and international restrictions. One of the greatest technical challenges in Phase Three (establishing the monitoring system) was the encounter with the strict sanctions imposed on container reference services, in particular Docker Hub.

6.4.1. Re-Engineering the Container Registry in an Isolated Environment

Because Docker Hub blocks the country's infrastructure IP addresses, the docker compose up command failed in the early stages with an access-denied error (HTTP 403 Forbidden). To resolve this and to allow the project to proceed, the workaround was reverse-engineered and implemented in two ways:

- Configuring the Docker engine to use domestic mirrors (Docker mirroring): by editing the Docker core configuration file at /etc/docker/daemon.json on Ubuntu, valid domestic alternative registries and mirrors were added.

Code Snippet 6.1: Setting the Docker engine to use domestic mirror servers

```
{
  "registry-mirrors": [
    "https://registry.docker.ir",
    "https://docker.arvancloud.ir"
  ]
}
```

- Managing the storage driver (the Overlay2 storage driver): owing to limitations of the Linux operating system in routing traffic within container environments, the loading of the heavy Zabbix database images occasionally encountered layer interruptions. By carefully setting

the Linux storage driver on Ubuntu to overlay2, the stability of the containers' filesystem was ensured during repeated reads from and writes to the physical disk [23].

6.5. How the Emulator Architecture (GNS3) Converges with Physical Production Environments

One of the key elements in evaluating computer-engineering theses is the analysis of the gap between the laboratory platform (the emulator) and real-world physical (production) environments. In this project, the Cisco operating systems were run through IOL (IOS on Linux) images within GNS3.

6.5.1. Examining the ASIC Hardware Layer versus CPU-Bound Processing

On physical Cisco devices (Catalyst switches), Layer 2 and Layer 3 packet processing is performed in hardware, on dedicated chips called ASICs (Application-Specific Integrated Circuits). ASICs are capable of switching frames at wire speed (line rate), with latency on the order of nanoseconds. In GNS3, however, these chips are absent, and all Layer 2 and Layer 3 processes are simulated in software by the CPU cores of the host system [6].

6.5.2. Technical Root-Cause Analysis of the Redundancy Protocol Bug (HSRP/VRRP Flapping)

In the initial redundant design of this project, a second distribution switch and a second (backup) router were deployed alongside swDistribution and R1 in order to provide first-hop gateway redundancy through HSRP/VRRP. The devices in each redundant pair shared a single virtual gateway IP address, one acting as the Active gateway and the other as the Standby, so that the failure of either device would not interrupt client connectivity. It was on this redundant pair that the flapping problem described below emerged. After the issue proved impossible to stabilise within the emulated environment, the redundant devices were removed and the topology was deliberately collapsed to the single-path design shown in Figure 3.1.

This structural difference (software versus hardware) was the principal cause of the errors encountered when implementing Layer 3 redundancy protocols such as HSRP and VRRP in this project. These protocols operate by transmitting continuous multicast hello packets at very short intervals (for example, every one second). In the emulator, when the Zabbix and Ansible

containers started up, the processing load on the Ubuntu Linux CPU increased. This introduced a delay of several milliseconds in the network packet-processing queue (CPU interrupts). On account of this momentary delay, the virtual IOL switches failed to receive one another's hello packets and erroneously concluded that the other switch had failed. This led to flapping (the repeated switching between the Active and Standby states) and to instability of the MAC address table. Consequently, in order to maintain the stability of the traffic baseline, these protocols were removed from the final configuration so that the Zabbix monitoring system could capture a stable picture of the network.

6.6. Risk Assessment, Threat Analysis, and Infrastructure Security

Hardening Strategies

A modern network infrastructure—even one that is configured automatically and possesses an excellent monitoring system—will be highly vulnerable to cyber threats in the absence of adherence to hardening principles. In this project, infrastructure security was introduced as a parallel layer across all phases, resulting in a defense-in-depth principle of applying multiple, independent security controls throughout the network rather than relying on a single point of protection [29].

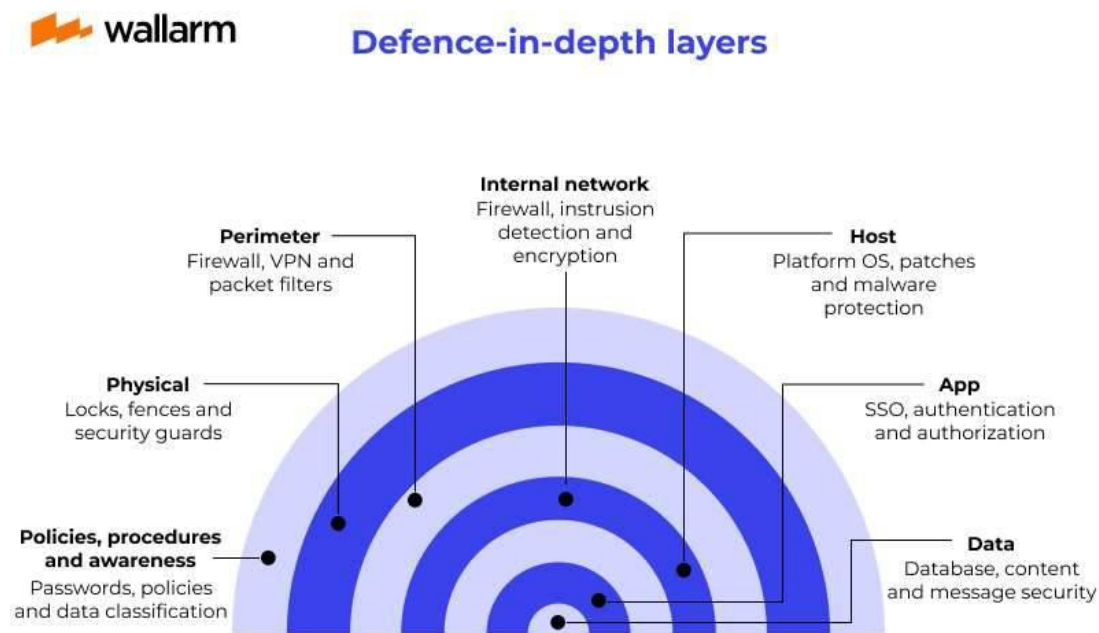


Figure 6.1. The layered security-hardening and defense-in-depth model implemented in the project [34]

6.6.1. Automating Device Access-Layer Security with Ansible

Using the Ansible automation playbooks from Phase Two, the process of hardening the devices' access lines (switches and the router) was executed automatically. The principal measures undertaken were as follows:

- Disabling insecure protocols: all ports associated with clear-text protocols such as Telnet and HTTP were blocked, and management access was restricted exclusively to the encrypted SSHv2 protocol (transport input ssh).
- Managing asymmetric keys: RSA keys (crypto key generate rsa modulus 2048) were generated on each device as a one-time manual prerequisite for SSHv2. Because key generation requires interactive confirmation and is not idempotent, it was deliberately kept outside the Ansible pipeline [35].

6.6.2. Vulnerability Analysis of the Monitoring Layer (SNMPv2c Mitigation)

The monitoring protocol used in this project is SNMPv2c. From a security standpoint, this version transmits the community string in clear text, rendering it susceptible to packet-sniffing attacks on the management network. To mitigate this risk, source-based restriction was implemented as a dedicated, one-time manual hardening step on each switch, separate from the repeatable Ansible pipeline. This separation reflects a deliberate design decision: the Ansible playbooks govern infrastructure state that evolves over time—VLAN assignments, security baselines, and monitoring preparation—whereas the SNMP ACL, bound permanently to the fixed IP address of the Zabbix server (192.168.10.20), is treated as immutable infrastructure policy. Keeping it outside the automation loop prevents the pipeline from inadvertently overwriting an operational security constraint during routine playbook re-runs, in keeping with the principle of separation of concerns between mutable and immutable configuration layers [7].

It is acknowledged that SNMPv3 affords a significantly stronger security posture through its support for authentication (HMAC-MD5 or HMAC-SHA) and privacy encryption (DES or AES) at the authPriv security level. The use of SNMPv2c in this project was a deliberate trade-off: the GNS3 IOL images used for emulation offer limited support for the full SNMPv3 USM (User-based Security Model) configuration workflow, and enforcing SNMPv3 across all devices would have diverted the focus of the research from the NetDevOps pipeline itself. The ACL-

based source restriction described above represents the principal compensating control within this laboratory environment. Migrating the Ansible playbooks and the Zabbix host configurations to SNMPv3 with authPriv is explicitly identified as a direct and high-priority avenue for future work.

6.7. Quantitative and Qualitative Analysis of the Monitoring Metrics and Operational Efficiency

The ultimate aim of combining automation (Ansible) with telemetry monitoring (Zabbix) is to improve the key performance indicators (KPIs) at the network-stability management layer. In traditional infrastructure management, error handling is reactive: the technician becomes aware of a problem only after users report it or a service outage is recorded. In this modern architecture, the network-management approach has been elevated to a proactive one [28].

6.7.1. Formulating and Analyzing the MTTR and MTBF Metrics

This project's efficiency can be analyzed through two key metrics in systems reliability engineering:

- **MTBF (Mean Time Between Failures):** the duration for which a system continues to operate without error. Through the application of security-hardening code and the elimination of configuration drift with Ansible, the stability of the devices increased and the rate of errors arising from human mistakes trends toward zero—the direct consequence of which is a notable increase in MTBF.
- **MTTR (Mean Time To Repair):** this metric measures the interval between the occurrence of an error and its complete resolution and return to normal. Its formula is as follows:

$$\text{MTTR} = \frac{\sum(\text{Time of Resolution} - \text{Time of Detection})}{\text{Total Number of Incidents}}$$

In the implemented model, the detection time was reduced to a few seconds owing to Zabbix's rapid polling over SNMP. Moreover, on account of the precise separation of severity levels and the escalation alerts, the technician identifies the technical root of the error within the first few seconds, and a substantial portion of the time traditionally expended on troubleshooting is eliminated—which sharply reduces the values in the fraction above and improves MTTR [36].

Table 6.2. Infrastructure-management metrics: the traditional model versus the modern model

Performance Metric	Traditional (CLI-Based)	Modern (NetDevOps)	Engineering Impact
Error detection	Reactive (user report or full outage)	Proactive (advanced Zabbix triggers and math functions)	Less downtime, SLA preserved
Speed of changes	Slow and manual (line-by-line in the terminal)	Instant and bulk (dynamic Ansible playbooks)	More agility in new segments
Config state of nodes	Asymmetric (drift and command conflicts)	Fully symmetric (consistency via code files)	Reduces configuration vulnerabilities by enforcing uniform, auditable baselines
MTTR	High (hours to find a link outage's root)	Very low (pinpoints the error in under 1 minute)	Backbone stability, less damage

6.8. Future Development Horizons, Part One: Implementing GitOps in Network Infrastructure Management

Although the automation in Phase Two ran successfully with Ansible, the playbooks are still initiated manually by the engineer through the terminal. One very appealing and practical development horizon for this design at scale is migrating to a GitOps architecture [37].

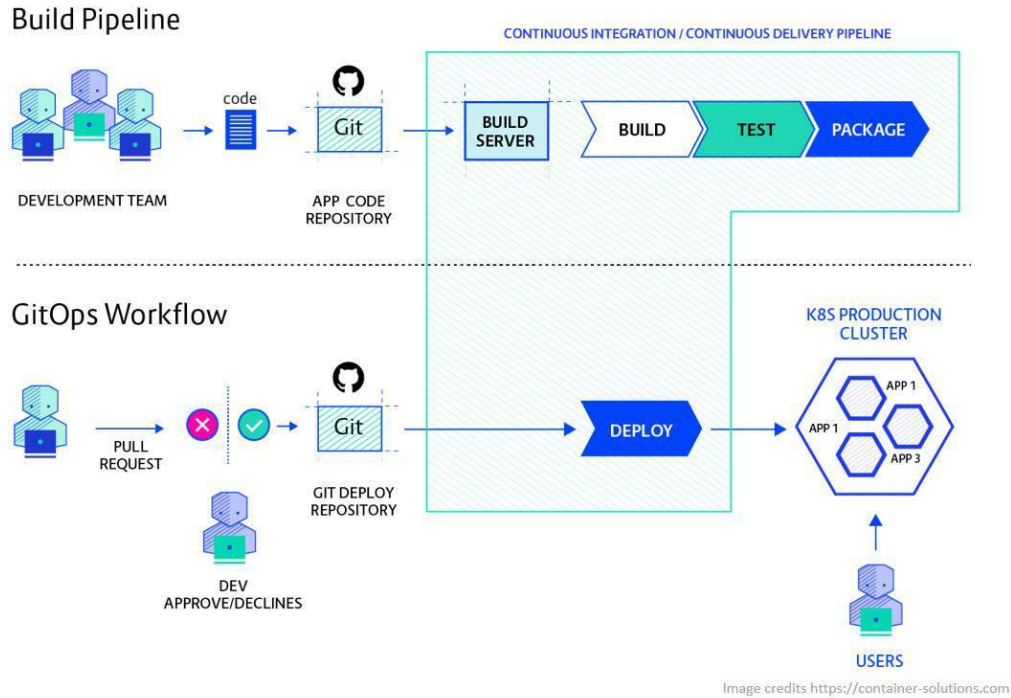


Figure 6.2. The GitOps pipeline workflow for automatically orchestrating network-device configuration.
(Image credit: Container Solutions, container-solutions.com)

6.8.1. Explaining the Single Source of Truth Concept

In the GitOps methodology, the Git repository (such as GitLab or GitHub) is treated as the single, definitive source of truth for the entire network. No engineer is permitted to connect directly to the switches' CLI and effect changes. Any change to the network's configuration (such as defining a new VLAN or changing an interface's IP address) is first recorded as a commit in the Ansible YAML files within Git [37], [38].

6.8.2. Orchestrating an Automated Validation Pipeline (CI/CD Pipeline)

After the changes have been committed to Git, a CI/CD tool (such as GitLab CI) is triggered automatically and validates the network code in a stepwise process before it is applied to the real devices. In this pipeline, `playbooks/site.yml` denotes a master playbook that sequentially imports the five operational playbooks developed in Chapter Four (`1_backup.yml` through `5_monitoring_prep.yml`) via Ansible's `import_playbook` directive, so that the entire configuration baseline can be validated and applied as a single, ordered unit.

Code Snippet 6.2: A GitLab CI pipeline for automatically validating the network Ansible playbooks

```
stages:
  - dry-run
  - deploy
network_simulation_check:
  stage: dry-run
  image: ansible/ansible-runner:latest
  script:
    - ansible-playbook --check playbooks/site.yml -i inventory.ini
  rules:
    - if: '$CI_PIPELINE_SOURCE == "merge_request_event"'
apply_network_config:
  stage: deploy
  image: ansible/ansible-runner:latest
  script:
    - ansible-playbook playbooks/site.yml -i inventory.ini
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
```

6.8.3. Technical Analysis of the GitOps Pipeline Steps

The pipeline above models two critical phases for rendering the network-development process safe:

- Phase one (dry run): to validate that the playbooks operate correctly before any real change is made to the live devices, the --check flag is used. This capability, known as dry-run mode, enables the engineer to predict and validate the likely output changes without altering the current configuration of the devices. This step prevents unwanted outages at the distribution layer [1].
- Phase two (deploy): only if the code passes the preceding dry-run stage and is merged into the main branch is it applied automatically to the real network hardware.

6.9. Future Development Horizons, Part Two: AI in Network Monitoring and Event Management (AIOps)

As networks grow and the volume of incoming metrics rises, analysis based on static thresholds loses its effectiveness. If a switch's CPU-usage threshold is set at 85%, the system raises repeated alarms during peak hours—even though such behavior may be entirely normal at that particular time. The final monitoring horizon for this project is to move beyond traditional monitoring and to enter the domain of AIOps (Artificial Intelligence for IT Operations) [39].

6.9.1. Dynamic Behavioral Monitoring Based on Machine Learning Algorithms (Baseline Detection)

More recent versions of Zabbix offer advanced functions for behavioral analysis (baseline monitoring). In the future development of this design, instead of setting numeric thresholds, Zabbix's machine-learning-based functions could be employed. By analyzing the Trends data over a window of three to six months, this system learns the network's behavioral pattern across the different days of the week and the different hours of the day.

For example, if the traffic on the swDistribution trunk link naturally decreases at weekends, the system stores this pattern as a baseline; should traffic then suddenly rise to 40% on a Friday (a figure entirely normal on ordinary days), the intelligent AIOps system recognizes this as anomalous behavior (anomaly detection) that probably indicates a cyberattack (such as a DDoS) or a failure at the access layer. This approach raises network monitoring to a new level of predictive intelligence (predictive telemetry) [21].

6.10. The Knowledge Management Approach, Code Structuring, and Engineering Documentation

In modern infrastructure engineering, documentation is not an ancillary process; it is one of the principal pillars of system stability. In traditional networks, network information (maps, IP addresses, and configurations) typically resides in the minds of technicians or in scattered text files, so that when a staff member departs, the technical knowledge departs with them. In this project, the knowledge-management approach was transformed in accordance with the Documentation as Code standard [40].

6.10.1. Self-Documenting Infrastructure with Declarative Code

By means of the Ansible and docker-compose files, the project's code itself serves as living documentation. Each access-layer switch (swEdu or swTech) is described precisely by the variables defined in the host_vars directory. Should a new engineer join the organization, there is no need to inspect each device individually; by reading the structure of the Ansible files, they obtain a complete understanding of the network's physical and logical layout. This transparency sharply reduces the risks of human error during infrastructure development.

6.11. Technical-Economic Analysis: Total Cost of Ownership (TCO) and Return on Investment (ROI)

A comprehensive engineering design must be economically sound for the organization, not merely technically performant. The implementation of the modern NetDevOps platform in this project was founded entirely on open-source tools such as Ubuntu Linux, Ansible, Docker, and Zabbix. This approach fundamentally alters the financial structure of network management in comparison with proprietary commercial solutions [41].

6.11.1. Total Cost of Ownership (TCO) Analysis

The total cost of owning the infrastructure comprises direct costs (the purchase of software and licenses) and indirect costs (maintenance, personnel, and service outages). If proprietary network-management and monitoring tools (such as SolarWinds or Cisco Catalyst Center) are used, the organization must pay annual licenses based on the number of ports or nodes (per-node licensing), which imposes a staggering cost on the IT budget. The use of open-source tools in this project reduced the software-license cost to zero.

6.11.2. Return on Investment (ROI) Calculations

The ROI metric in infrastructure engineering indicates how profitable or cost-saving a technology is relative to the expenditure incurred on it. Its formula is as follows:

$$\text{ROI} = \frac{\text{Financial Benefit (Savings)} - \text{Implementation Cost}}{\text{Implementation Cost}} \times 100\%$$

In this project, the factors in the numerator (financial benefit) include:

- a 100% saving from forgoing the purchase of commercial monitoring licenses;
- a significant reduction in downtime costs, owing to the lower MTTR afforded by the Zabbix triggers;
- less time expended by personnel in configuring devices, owing to the Ansible playbooks (lower operating expenditure, OpEx).

Given that the denominator (the implementation cost) here amounted only to the cost of the monitoring-server hardware (which, thanks to Docker containers, can run on an ordinary server) together with the engineering time, a precise ROI figure was not computed in this study; however, the elimination of licensing costs and the reduction in operational overhead align with the general pattern of positive returns reported for DevOps adoption in enterprise settings [41].

Table 6.3. Financial comparison of the NetDevOps platform versus proprietary models

Financial Component	Proprietary / Commercial (e.g., SolarWinds)	Implemented NetDevOps Platform	Economic Justification
Initial purchase cost (CapEx)	Very high (annual, dollar-priced licenses)	Zero (open-source, free platforms)	Eliminates foreign-currency software costs
Vendor lock-in	Complete (locked to one protocol or brand)	None (broad support for standard equipment)	Freedom to choose new equipment
Host hardware cost	High (heavy dedicated servers, Windows licenses)	Optimal (Docker + Ubuntu, minimal hardware)	Lower hardware infrastructure costs
Scaling cost (Expansion)	Exponential (every new switch raises license cost)	Linear and zero (add unlimited nodes)	Foundation for unlimited growth

6.12. Final Summary, High-Level Evaluation of the Achievements, and Closing Words

This thesis was designed and implemented in a stepwise manner, with the aim of moving from traditional, obsolete network management toward modern paradigms and code-based infrastructure automation (NetDevOps). The life cycle of this engineering project began with the initial phase of designing the Cisco three-layer topology, in which the network’s physical and logical structure was established in GNS3 on the basis of the principles of stability and of separating the core, distribution, and access layers.

In the second phase, through the introduction of the powerful Ansible tool, an agentless automation pipeline took shape, and configuration code—as the source of truth—replaced CLI commands. This step enhanced the operational agility of the infrastructure. Finally, through the deployment of the containerized Zabbix monitoring platform on Docker in the third phase, comprehensive, real-time observability was established across all network layers, so that resource monitoring, interface-traffic analysis, and stepwise error alerting could emerge in a fully proactive manner.

Although challenges such as the international container sanctions and the processing limitations of the emulator environment (which prevented the gateway-redundancy protocols HSRP and

VRRP from running stably) arose in the course of the work, the reverse-engineering solutions applied and the critical analyses conducted ensured the stability of the system.

The results of this research demonstrate that the convergence of DevOps concepts and network engineering not only sharply reduces the total cost of ownership (TCO) and maximizes the return on investment (ROI), but also provides a stable, scalable, and secure infrastructure that is fully prepared to enter the era of software-defined networking (SDN) and of AI-based predictive monitoring (AIOps). This achievement constitutes a firm step toward intelligent, automated, and highly stable networks within the structure of modern organizations.

References

- [1] J. Edelman, C. Adell, S. Lowe, and M. Oswalt, *Network Programmability and Automation: Skills for the Next-Generation Network Engineer*. O'Reilly Media, 2023.
- [2] A. Clemm, *Network Management Fundamentals*. Cisco Press, 2006.
- [3] D. Oppenheimer, A. Ganapathi, and D. A. Patterson, "Why Do Internet Services Fail, and What Can Be Done About It?" in *4th USENIX Symposium on Internet Technologies and Systems (USITS 03)*, 2003.
- [4] P. Oppenheimer, *Top-Down Network Design*, 3rd ed. Cisco Press, 2010.
- [5] D. Teare, B. Vachon, and R. Graziani, *Implementing Cisco IP Routing (ROUTE) Foundation Learning Guide*. Cisco Press, 2015.
- [6] J. Neumann, *The Book of GNS3: Build Virtual Network Labs Using Cisco, Juniper, and More*. No Starch Press, 2015.
- [7] K. Morris, *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media, 2016.
- [8] L. Hochstein and R. Moser, *Ansible: Up and Running*, 3rd ed. O'Reilly Media, 2022.
- [9] B. Ratan, *Practical Network Automation*. Packt Publishing, 2017.
- [10] R. Perlman, *Interconnections: Bridges, Routers, Switches, and Internetworking Protocols*, 2nd ed. Addison-Wesley, 2000.
- [11] O. Santos and J. Stuppi, *CCNA Security 210-260 Official Cert Guide*. Cisco Press, 2015.
- [12] W. Odom, *CCNA 200-301 Official Cert Guide, Volume 1*. Cisco Press, 2019.
- [13] K. Dooley and I. Brown, *Cisco Cookbook*. O'Reilly Media, 2003.
- [14] W. Odom, *CCNP Enterprise Core ENCOR 350-401 Official Cert Guide*. Cisco Press, 2020.
- [15] J. Geerling, *Ansible for DevOps*, 2nd ed. Midwestern Mac, LLC, 2020.
- [16] E. Chou, *Mastering Python Networking*, 4th ed. Packt Publishing, 2023.
- [17] Y. Bhaiji, *Network Security Technologies and Solutions (CCIE Professional Development Series)*. Cisco Press, 2008.
- [18] A. Dalle Vacche, *Mastering Zabbix*, 2nd ed. Packt Publishing, 2015.
- [19] B. Brazil, *Prometheus: Up & Running*. O'Reilly Media, 2018.
- [20] D. Josephsen, *Building a Monitoring Infrastructure with Nagios*. Prentice Hall, 2007.
- [21] N. Liefing and B. van Baekel, *Zabbix 6 IT Infrastructure Monitoring Cookbook*. Packt Publishing, 2022.
- [22] J. Nickoloff and S. Kuenzli, *Docker in Action*, 2nd ed. Manning Publications, 2020.
- [23] Docker, Inc., "Configure the Docker daemon," *Docker Documentation*. [Online]. Available: <https://docs.docker.com/engine/daemon/>. [Accessed: Sep. 2024].
- [24] D. Mauro and K. Schmidt, *Essential SNMP: Help for Network Administrators*, 2nd ed. O'Reilly Media, 2005.

- [25] W. Stallings, SNMP, SNMPv2, SNMPv3, and RMON 1 and 2, 3rd ed. Addison-Wesley Professional, 1999.
- [26] GeeksforGeeks, "Simple Network Management Protocol (SNMP)," GeeksforGeeks, Oct. 2025. [Online]. Available: <https://www.geeksforgeeks.org/computer-networks/simple-network-management-protocol-snmp/>. [Accessed: Jun. 2026].
- [27] C. Majors, L. Fong-Jones, and G. Miranda, Observability Engineering: Achieving Production Excellence. O'Reilly Media, 2022.
- [28] M. Julian, Practical Monitoring: Effective Strategies for the Real World. O'Reilly Media, 2017.
- [29] O. Santos, End-to-End Network Security: Defense-in-Depth. Cisco Press, 2007.
- [30] J. Turnbull, The Docker Book: Containerization is the New Virtualization. James Turnbull, 2014.
- [31] P. Roethlisberger, Zabbix Network Monitoring Essentials. Packt Publishing, 2015.
- [32] R. Olups, Zabbix Network Monitoring, 2nd ed. Packt Publishing, 2016.
- [33] Zabbix LLC, "History and Trends," Zabbix 6.4 Documentation. [Online]. Available: https://www.zabbix.com/documentation/6.4/en/manual/config/items/history_and_trends. [Accessed: Sep. 2024].
- [34] M. Beschokov, "What Is Defense in Depth? Strategy for Cybersecurity," Wallarm Learning Center, Jul. 2025. [Online]. Available: <https://www.wallarm.com/what/defense-in-depth-concept>. [Accessed: Jun. 2026].
- [35] W. Stallings, Cryptography and Network Security: Principles and Practice, 8th ed. Pearson, 2020.
- [36] B. Beyer, C. Jones, J. Petoff, and N. R. Murphy, Site Reliability Engineering: How Google Runs Production Systems. O'Reilly Media, 2016.
- [37] B. Yuen, A. Matyushentsev, T. Ekenstam, and J. Suen, GitOps and Kubernetes: Continuous Deployment with Argo CD, Jenkins X, and Flux. Manning Publications, 2021.
- [38] Microsoft, "Deploying with GitOps," ISE Engineering Fundamentals Playbook, Aug. 2024. [Online]. Available: <https://microsoft.github.io/code-with-engineering-playbook/CI-CD/gitops/deploying-with-gitops/>. [Accessed: Jun. 2026].
- [39] Y. Dang, Q. Lin, and P. Huang, "AIOps: Real-World Challenges and Research Innovations," in 2019 IEEE/ACM 41st International Conference on Software Engineering: Companion Proceedings (ICSE-Companion), 2019, pp. 4-5.
- [40] A. Gentle, Docs Like Code, 2nd ed. Lulu.com, 2017.
- [41] N. Forsgren, J. Humble, and G. Kim, Accelerate: The Science of Lean Software and DevOps. IT Revolution Press, 2018.